

Applicability Is Not Competitiveness: An Independent Evaluation and a Data-Side Repair for BQ-Native Graph Indexing

Chengcheng Li (Beyondsoft), qq1330494624@outlook.com

October 9, 2026

Abstract

Binary-quantized graph indexes navigate on 2-bit codes instead of full-precision vectors, and the published answer to “when is that usable?” is a 12-dataset table spanning 0.40 % $\rightarrow$ 95.65 % Recall@10 plus a single index-free compatibility probe. We re-run that benchmark on the authors’ released implementation (11 of 12 rows reproduced within ± 1.84 pp; one row is protocol-ambiguous) and report five things the published account does not cover. (i) The table’s tiers describe applicability, and are read as competitiveness; where competitor curves exist, three of the four tiers are dominated by plain HNSW — on several rows QuIVer’s recall ceiling lies below the baseline’s lowest operating point, so the curves cannot intersect. (ii) The bottom tier mixes three different failures: a repairable encoding failure (the sign plane is globally constant, sign_info = 0.000), an index-agnostic task failure on which every competitor also collapses (Random-Sphere 1.40 %, Gaussian-960 1.10–1.39 % across three HNSW implementations at ef=64), and a capacity failure (coco_nomic) where the bit budget, not the distribution, is binding — at 4 bits per dimension every collapse row measures ≥ 94.6 %. (iii) The published probe is under-specified — it does not fix the BQ distance, the sample size, or the instrument. Implemented with the paper’s own default distance, it calls Random-Sphere compatible (53.9%) on a dataset whose measured recall is 0.91 %; taking the weaker of the engine’s two distances removes this false positive without changing any other verdict, and a sign-entropy statistic paired with it yields a four-step decision chain validated on 22 arms. (iv) The collapse is repairable without touching the index: one translation ($x' = \text{normalize}(x - \mu)$) takes GIST-960 from 2.10 % to 39.74 % and SIFT-128 from 15.77 % to 30.64 % at ef=64, with an isotropic control moving +0.03 pp; making the L0 navigation metric match the metric the graph was built with adds up to +21.8 pp where the sign plane is alive and hurts where it is dead, so we ship it as a default-off switch with a decision rule. After the repair two arms move from “dominated” to “parity” or “intersecting” — but the honest sentence survives: a smaller gap is not availability. (v) The missing baseline class (§5.6): an OPQ+IVF-PQ+Refine pipeline with the same f32 re-ranking never collapses on any of the six cells we measured — including the two where the 2-bit code collapses hardest — at $\approx 4.7\times$ QuIVer’s memory, so the published boundary is the 2-bit code’s, not quantization’s. We do not claim the paper is wrong (its mechanism is the one we measure), we propose no new quantizer, and we report our own two failed attempts at a single-scalar recall predictor.

Contents

1. Introduction	3
2. Background and Notation	4
2.1 QuIVer’s 2-bit encoding	4
2.2 What the paper already establishes	4
2.3 Baselines	5
2.4 Relationship to rotation-based quantizers	5
3. Methodology	5
3.1 Harness and equivalence	5
3.2 Metrics	5
3.3 Configuration	6
3.4 Protocol deviations (disclosed, and why they do not affect comparisons)	6
3.5 Reproducibility and noise floor	6

3.6 Index-free instruments	6
3.7 How a competitiveness verdict is reached	6
4. Reproduction on the Published Cross-Dataset Benchmark	6
4.1 What we reproduce, and why	7
4.2 Setup, and the one protocol deviation	7
4.3 Result: all twelve rows reproduce	7
4.4 The one row that needs a stated assumption: RedCaps-1M	8
4.5 Matched-recall speedups	8
4.6 Four cross-checks beyond Table 11	9
4.7 What Sec. 4 establishes	9
5. Applicability Is Not Competitiveness	9
5.1 The question a deployer actually asks	10
5.2 Method: a criterion that does not depend on QPS	10
5.3 Result: three of the four tiers are dominated	10
5.4 A protocol-level caveat worth carrying forward: the tie floor	10
5.5 Implication	11
5.6 Is that boundary a property of the data, or of the code?	11
6. Diagnosis: Why the Code Fails, and Three Different Failures	13
6.1 The mechanism, in one identity	13
6.2 Quantifying “alive”: <code>sign_info</code>	13
6.3 The graph is faithful to the wrong metric	13
6.4 Three different failures inside the paper’s “< 15 %” tier	14
6.5 The axis that actually moves the outcome: bits per dimension	15
7. The Judge: Three Specification Gaps, and a Repaired Chain	15
7.1 What is specified, and what is not	15
7.2 What the gaps cost (measured)	16
7.3 The repaired chain	16
7.4 Our own two failed predictors (reported as such)	18
8. A Two-Step Data-Side Repair	18
8.1 Step 1 — centre: <code>x' = normalize(x - mu)</code>	18
8.2 What centring does, tier by tier	19
8.3 Step 2 — use the metric the graph was built with (data-dependent)	19
8.4 The competitor map: what the repair does and does not buy	19
8.5 What this repair is not	21
8.6 A second repair, and on the collapse tier a better one: a seeded rotation	21
8.7 Is BQ-native still the right choice? The quantizer arm, and what the repair costs in memory	23
9. Discussion	24
9.1 The decision a deployer actually faces	24
9.2 Translation or rotation? The head-to-head	24
9.3 What this says about the published applicability claim	24
9.4 A methodological note we owe the reader	25
9.5 Does the boundary appear outside the paper’s twelve datasets? (VIBE)	25
10. Limitations	26
10.1 Platform and protocol	26
10.2 The reproduction has one unresolved row, one tie floor	26
10.3 Experimental scope	27
10.4 Statistical conventions used throughout	28
11. Artifacts and Reproduction	28
11.1 Index-side harness (Rust bench, no library changes except the Sec. 8.3 switch)	28

11.2 Data preparation and analysis scripts (Python; <code>src/</code> untouched)	28
11.3 Where each table comes from	29
11.3a Memory footprint (1 M x 768, M = 32)	29
11.4 One-command reproduction of the central claim	30
11.5 Upstream material (parallel track)	30
11.6 Artifacts added after the first draft (second repair, quantizer arm, external validity)	30

12. References, Artifact Statement, and How to Assemble This Draft	31
12.1 The object of study	31
12.2 Baseline implementations we measured against	31
12.3 Background and method references	32
12.4 Artifact statement (what is ours, what is upstream, and the licence)	32

Evidence base. Every number in this paper resolves to a file under `results/**` or a log under `results/logs/`; the audit trail — including our own conclusions that were retracted along the way, and the protocol correction of §5.6(b) — is in `docs/research/*.md`. The only library change is a default-off switch shipped as `patches/f1-nav-weighted.patch`.

1. Introduction

Binary-quantized (BQ) graph indexes promise a graph that navigates on 2-bit codes instead of full-precision vectors: an order of magnitude less memory, and a hot path that is mostly popcount. Whether such an index is *usable* on a given embedding distribution is therefore a question with real deployment consequences — and it is the question the QuVer paper (PVLDB 20) answers with a cross-dataset table spanning more than 90 points of Recall@10 (0.40 % → 95.65 %) and a single index-free probe that predicts which side of the usability line a dataset is on.

This paper is an independent evaluation of that claim, plus a constructive repair of its two weakest points. We re-run the published benchmark on the released implementation, on a machine without AVX-512, and report what survives, what does not, and what a deployer should do instead.

What we find.

1. **All twelve rows reproduce** (eleven unconditionally within ± 1.84 pp; the RedCaps row only under an assumed sampling protocol, whose four plausible readings span 7.4 pp). Our numbers come from the authors’ own index driven by our harness, with all arms in a single process — so the table in §4 is comparable value-for-value with the published one. This is the foundation for everything else.
2. **The published tiers describe *applicability*, and are routinely read as *competitiveness* — they are not the same thing (§5).** Where competitor curves exist, three of the four tiers are dominated by plain HNSW, including the tier the paper labels “moderate”. The judgement needs no QPS argument: on several rows QuVer’s recall *ceiling* sits below the baseline’s *lowest* operating point, so the curves never intersect.
3. **The bottom tier contains three different failures (§6).** Some datasets collapse because the 2-bit code is asked to do something it cannot do (their sign plane is constant, `sign_info` = 0.000) — an *encoding* problem. Others collapse because the task has no usable neighbourhood structure at all: on Random-Sphere, HNSW also collapses (1.40 % at `ef=64`), and on Gaussian-960 all three HNSW implementations and IVF-Flat do (1.10–2.41 %). A third fails on code *capacity* alone — the VIBE benchmark `coco_nomic` is the clean example (§9.5) — and here the fix is a different code: at 4 bits per dimension every collapse row measures ≥ 94.6 % (§6.5). The paper answers all three with “use float32”, which hides that the first is repairable, the second is not, and the third is repairable only by changing the code.
4. **The probe is under-specified, and the gaps have consequences (§7).** §6 of the paper does not say which of the engine’s two BQ distances to use, how many samples, or which instrument. Implemented with the paper’s own default metric, the probe calls Random-Sphere *compatible* (53.9%) on a dataset whose measured recall is **0.91** %. Taking the weaker of the two metrics fixes this without changing any other verdict, and pairing it with a sign-entropy statistic yields a four-step decision chain that we validate on 22 arms — including a **6/6 prospective hold-out** frozen before those six arms were measured (§7.3).

5. **The collapse is repairable by a two-step, data-side intervention (§8).** One translation ($\mathbf{x}' = \text{normalize}(\mathbf{x} - \mu)$) moves GIST-960 from 2.10 % to 39.74 % and SIFT-128 from 15.77 % to 30.64 % at $\text{ef}=64$, with an isotropic control that moves by **+0.03 pp**; making the L0 navigation metric match the one the graph was built with adds up to **+21.8 pp** on rows whose sign plane is alive — and *hurts* rows whose sign plane is dead, which is why we ship it as a default-off switch with a decision rule. After the repair, two arms move from “dominated” to “parity” or “intersecting” — but the paper’s honest sentence survives: **a smaller gap is not availability**. Where the sign plane is *dead*, a **seeded random rotation** — which leaves the similarity function untouched (GT overlap 99.87–100 %) — does better still (**60.22 % / 60.24 %** at $\text{ef}=64$) and turns GIST-960 into a *win* ($1.3\times$ faster than HNSW at 84 % recall, and the only measured option in the 60–84 % recall band); on a live sign plane it hurts (Cohere -4.8 pp), so the same statistic decides which of the two repairs to use.
6. **The boundary is the code’s, not the data’s — and a different code steps around it (§5.6, §8.7).** Table 11 carries no quantizer baseline; on all six cells we measured, an OPQ+IVF-PQ+Refine pipeline with the same f32 re-ranking never collapses — 98.99 % / 98.42 % on the two tasks where the 2-bit code collapses hardest — at $\approx 4.7\times$ QuIVer’s memory. QuIVer keeps the win at the competitive tier ($3.7\times$ at ≥ 99 % recall). The same measurement also surfaced a trap that can fake such a verdict — a cosine ground truth on raw vector files — which we diagnose and document rather than hide (§5.6b).
7. **Two engineering facts the paper does not report (§8.3, §9.2):** the L0 query navigation uses a *different* distance than build/prune/upper layers, and the α default sits at the worst end of its own platform.

What we do not claim. We do not claim the paper is wrong: its mechanism (Finding 1 — the sign plane of non-negative embeddings carries no information) is the same mechanism we measure, and we reproduce it. We do not propose a new quantizer, and we do not claim a single scalar can predict recall: our two attempts to build one failed and are reported as failures (§7.4). The contribution is a deployment-oriented *triage rule*, a *competitiveness* boundary the published table does not draw, and a quantified repair path.

Roadmap. §2 fixes notation; §3 the harness and protocol; §4 the reproduction; §5 competitiveness; §6 diagnosis; §7 the probe and our repaired chain; §8 the repair and its competitor consequence; §9 discussion; §10 limitations; §11 artifacts.

2. Background and Notation

2.1 QuIVer’s 2-bit encoding

Each vector $\mathbf{v} \in \mathbb{R}^d$ (assumed unit-norm) is encoded into two bit planes:

```
pos(v)      = (v_j > 0)                the sign plane
strong(v)   = (|v_j| > mean_j |v_j|)    the magnitude plane
```

The engine’s distance for build/prune/upper layers is the 6-class **weighted** distance; its L0 query navigation uses a plain-Hamming **cheap** distance. We use the following two forms throughout, and verified them against `src/index/bq.rs` bit-for-bit (`bq2_code_ceiling.py::verify_identities`; asserted `max |analytic - bitwise| = 0`):

```
pos      : p = (v > 0)                (strict => 0 falls in the negative half-plane)
strong   : s = (|v| > mean|v|)         (per-vector threshold tau)
cheap    : D(a,b) = (|p_a| + |s_a|) - 2(<p_a,p_b> + <s_a,s_b>)    ascending = nearer
weighted: S(a,b) = <w_a, w_b>,      w = (2p - 1)*(1 + s) in {-2,-1,+1,+2}    descending = nearer
```

The identity $\mathbf{w} = (2\mathbf{p}-1)(1+\mathbf{s})$ is the reason the sign plane matters so much: if $\mathbf{p} \equiv 0$ then $\mathbf{w} = -(1 + \mathbf{s})$ and the score degenerates into a function of the *magnitude* statistics of the two codes (§6.1). This is a restatement of the paper’s Finding 1 in closed form, not a new finding.

2.2 What the paper already establishes

The paper’s statement	Our position
Finding 1: recall is governed by the data distribution, not by dimension; on SIFT/GIST the values lie in a narrow positive band, so the sign plane loses discriminative power	We reproduce and agree. We quantify it as <code>sign_info</code> (§6.2) and use it as a triage quantity; we claim no priority.
Finding 2: no hard recall ceiling; recall rises monotonically with <code>ef</code> on all 12 datasets	Agreed. Our earlier “SIFT ceiling 47.21 %” phrasing referred to a <i>practical</i> <code>ef</code> budget and is withdrawn.
Finding 3: applicability is a continuous gradient, reported as four tiers (<15 %, 32–42 %, 71–78 %, >88 %)	Agreed as <i>description</i> . Our §5 shows the tiers are not a competitiveness ordering.
Finding 4: two necessary conditions — low effective rank (to create a detectable angular gap) and contrastive geometry (to organise the gap into semantic neighbourhoods)	Agreed; our Random-Sphere (§6.4) is the same phenomenon, now with HNSW as an external witness.
§6 Practical compatibility test: ≈ 10 K samples, BQ-ranked vs float32-ranked top-K overlap, $> \sim 50$ % $\Rightarrow$ compatible	We implement it as written and report the three specification gaps and their measured cost (§7).

2.3 Baselines

We measure, on every task, the same five reference points in a single process: **hnswlib**, **FAISS-HNSW**, **USe-arch**, **FAISS IVF-Flat**, and **faiss_exact** (exact float32 search). The last one is not a baseline but a *protocol instrument*: it reveals tie floors. On Wolt-CLIP-1M it scores only **90.09 %**, so that dataset’s recall cannot be compared with datasets whose exact search returns 99.9–100 %.

2.4 Relationship to rotation-based quantizers

Rotation-based quantizers (randomised rotations as in RaBitQ, learned rotations as in OPQ, product quantization) address the *anisotropy* of the embedding cloud, and a rotation would also revive a constant sign plane. We do not claim that translation (“centring”) dominates them, and we did not run the head-to-head (§9.2, L12). We choose translation for a different reason: it is the minimal intervention whose **applicability is decided by the same statistic that detects the problem** — and it requires no rotation matrix, no training, and no change to the index. The retrievability post-processing literature (all-but-the-top family) has used centring for other purposes; our contribution here is the decision rule and the quantification, not the transform.

3. Methodology

3.1 Harness and equivalence

All index-side numbers come from two benches we added on top of the authors’ released code:

- `benches/bench_t2_b2_partitioned.rs` — the measurement arm. It supports a **single-arm mode** (one index, recall + MT-QPS + build wall-clock) and an ablation mode (several arms in one process). Both modes go through the library’s public search path; we verified equivalence against the repository’s own evaluation entry point on shared configurations before using either. All arms of a comparison run in **one process** with the same build flags, so cross-arm differences are not confounded by rebuild or run-to-run variance.
- `benches/bench_t2_build_recon.rs` — exports the L0 adjacency as CSR for the graph-fidelity measurements (§6.3).

For every graph arm we additionally build the competitor indexes on the *same* base and query set in the same machine (hnswlib, FAISS-HNSW, USearch, IVF-Flat) and record exact search (**faiss_exact**) as the ceiling probe.

3.2 Metrics

- **R@10** — Recall@10 against the **exact float32 top-10 of the same base and query set** (recomputed by us under the same similarity function as the arm under test). We report it at `ef_s` $\in$ {64, 128, 256, 512, 1024}.
- **MT-QPS** — multi-threaded throughput at 32 threads (and a 16-thread parity run, matching the paper’s thread count, to check that the same-recall speed ratio is thread-count independent).
- **Build wall-clock** — reported because it is one of the costs of the repair path.

- **faiss_exact** — used as a *tie floor* instrument, not as a baseline: if exact search cannot reach 100 % on a dataset (Wolt-CLIP: 90.09 %), the dataset has many exact ties and its recall must not be compared across datasets (L6).

3.3 Configuration

$m = 32$, $ef_c = 128$, $\alpha = 1.2$ (the paper’s configuration throughout), `--features ablation`, `RUSTFLAGS="-C target-cpu=native"`, release profile with LTO (inherited from `[profile.release]`), Windows 11, Intel i9-14900K (24 C/32 T, 63.7 GB, **no AVX-512**).

3.4 Protocol deviations (disclosed, and why they do not affect comparisons)

1. **Similarity function.** The public SIFT-128/GIST-960 benchmarks are Euclidean; the repository’s own `prepare_all.py` L2-normalises those vectors and **recomputes** ground truth by cosine. We follow the repository, so our SIFT/GIST rows are cosine tasks. Every *comparison* we report (tier $\times$ competitor, before/after repair, metric A/B) puts both sides on the **same** task, so the deviation cancels; only absolute cross-paper values are affected (L2).
2. **Platform.** No AVX-512, 32 threads. Recall-level conclusions are platform-independent; for speed we report same-recall ratios, and we note that our absolute MT-QPS already exceeds the paper’s (1.1–2.4 $\times$), so we are not measuring on a weakened platform (L1).

3.5 Reproducibility and noise floor

Graph construction is concurrent, so the L0 edge set is not bit-reproducible. We built and measured five tier representatives **three times each**; the spread of $R@10$ @ $ef=64$ is ≤ 0.25 pp (GIST-960 0.22, SIFT-128 0.25, GloVe-100 0.10, Wolt-CLIP 0.19, Cohere 0.16; at $ef_s = 1024$ the spread stays ≤ 0.27 pp). The per-seed values and spreads are in `results/t2/p7_stagec_report.json` (raw logs `results/logs/p7q*_seed{1,2,3}.log`). We therefore (a) treat differences below ~ 0.3 pp as unresolved, and (b) mark any graph-structure quantity as a single-sample estimate. For the arms where a **src/** change is evaluated (§8.3) we use **frozen recall values as a regression guard**: with the switch off, the frozen values reproduce within ≤ 0.17 pp.

3.6 Index-free instruments

The gate (§7) and the probe analysis use quantities that need no index:

- **sign_info** = $\text{mean}_j H(p_j)$ with p_j the fraction of negative coordinates in dimension j , H the binary entropy in bits (§6.2);
- $\|\mu\|$ — norm of the mean of the **normalised** training vectors (with the ≤ 1 sanity guard of §8.1);
- **probe10** — overlap of the code’s top-10 with the exact float32 top-10 (the paper’s literal instrument);
- **probe_ef** — take the code’s top-128 candidates, re-rank them in float32, take the top-10, and measure overlap with the exact top-10 (the instrument that matches how a graph search actually behaves, §7.2c). Both probes are computed under **both** BQ metrics and we report the weaker one.

3.7 How a competitiveness verdict is reached

We avoid QPS-dependent arguments where possible. A tier is **dominated** if QuVer’s recall *ceiling* (its best point over the whole ef_s sweep, including the pipeline of §8) lies **below** the competitor’s *lowest* operating point on the same task — in that case the curves cannot intersect, and no throughput measurement can rescue the tier. Otherwise we report the same-recall throughput ratio and call the outcome **parity** or **intersecting**.

4. Reproduction on the Published Cross-Dataset Benchmark

Notation: $R@10$ is Recall@10 against the **exact float32 top-10 of the same base and query set**; ef_c is the construction beam width, ef_s the search beam width; *MT-QPS* is multi-threaded throughput.

4.1 What we reproduce, and why

The applicability claim of QuVer is the most consequential part of the paper for a practitioner: it states *which* embedding distributions a BQ-native graph may be used on, and encodes that statement in a single cross-dataset table (Table 11) whose values span more than 90 points of Recall@10 (0.40 % $\rightarrow$ 95.65 %). Everything we do in §5–§8 builds on that table, so we first establish that we can reproduce it.

We reproduce it by re-running the measurement, not by re-deriving it: the index under test is the released implementation in the authors’ repository, driven from a bench harness that we added (`benches/bench_t2_b2_partitioned.rs`) and that we verified against the repository’s own evaluation paths. All arms in a comparison run in a **single process** with the same build flags, so cross-arm differences are not confounded by run-to-run variance.

4.2 Setup, and the one protocol deviation

item	value
Machine	Intel i9-14900K, 24C/32T, 63.7 GB RAM, Windows 11 Pro
SIMD	AVX-512 unavailable (<code>Avx512F = false</code>) — the VPOPCNTDQ path QuVer is designed around is <i>not</i> exercised
Build	<code>RUSTFLAGS="-C target-cpu=native", --features ablation</code>
Parameters	<code>m = 32, ef_c = 128, $\alpha = 1.2$</code> (the paper’s configuration)
Metrics	R@10; MT-QPS; build wall-clock
Datasets	the 12 rows of Table 11: 9 automated (<code>prepare_all.py</code>), 2 synthetic generators (<code>bench_randomlm</code> , <code>bench_random_sphere</code>), RedCaps (§4.4)

Deviation (disclosed). For the ann-benchmarks sources the published metric is *Euclidean* (SIFT-128, GIST-960). The repository’s own `prepare_all.py` L2-normalizes those vectors and **recomputes** ground truth by cosine, so our SIFT/GIST rows are *cosine* tasks rather than the published Euclidean tasks. Our numbers are therefore comparable to the paper’s *pipeline* but not to the public ann-benchmarks Euclidean leaderboard; we return to this in §10.

Reproducibility of a single arm. Graph construction is concurrent, so the L0 edge set is not bit-reproducible between runs. To bound what that costs, we re-built and re-measured the five tier representatives three times each: the spread of R@10 @`ef = 64` is $\leq$ **0.25 pp** (GIST-960 0.22, SIFT-128 0.25, GloVe-100 0.10, Wolt-CLIP 0.19, Co-here 0.16; at `ef_s = 1024` the spread stays $\leq$ 0.27 pp; per-seed values in `results/t2/p7_stagec_report.json`). We therefore treat recall-level conclusions as comparable at that resolution, and mark any graph-structure quantity as a single-sample estimate.

4.3 Result: all twelve rows reproduce

Dataset	Dim	Tier (paper)	Ours R@10 @ <code>ef=64</code>	Paper (Table 11)	Δ
Cohere-1M	768	competitive	94.63 %	95.13 %	−0.50
MiniLM-1M	384	competitive	88.94 %	88.09 %	+0.85
BGE-M3-1M	1024	competitive	94.91 %	93.81 %	+1.10
DBpedia-1M	1536	competitive	94.64 %	95.34 %	−0.70
DBpedia-3072	3072	competitive	95.70 %	95.65 %	+0.05
Wolt-CLIP-1M	512	moderate	71.48 %	70.68 %	+0.80
RedCaps-1M	512	moderate	77.08 % †	78.41 %	−1.33
SIFT-128	128	collapse	15.77 %	14.85 %	+0.92
GIST-960	960	collapse	2.10 %	2.01 %	+0.09
GloVe-100	100	usable	32.82 %	32.08 %	+0.74
Synthetic-LR	768	usable	43.60 %	41.76 %	+1.84
Random-Sphere	768	collapse	0.48 %	0.40 %	+0.08

† conditional on a protocol we had to infer; see §4.4.

Eleven rows lie within ± 1.84 pp and ten within ± 1.1 pp, across 100–3072 dimensions, four data sources and five orders of magnitude of recall. We also reproduce the *rank order* of the four tiers exactly. This is the basis for

treating Table 11 as a faithful description of the released implementation, and for asking in §5 what the tiers actually mean for a deployer.

4.4 The one row that needs a stated assumption: RedCaps-1M

The repository’s reproduction guide names the source explicitly (Zenodo record 13137120, “CLIP-Embedded Red-Caps Text-Image Dataset”, CC BY 4.0); we downloaded it and verified the published MD5 (a6221cc0a4103af7e0f06f87bd989a022.12 GiB). The artifact contains

key	shape	note
<code>train</code>	(11,588,824, 512)	all rows already L2-unit-norm
<code>random_test</code>	(10,000, 512)	unit-norm
<code>test</code>	(800, 512)	unit-norm; max cosine to the base only
—	—	0.27–0.38, i.e. a <i>different</i> kind of query
		no neighbors / distances $\Rightarrow$ ground truth must be recomputed

The guide specifies the output shapes (1 M $\times$ 512 base, 10 K $\times$ 512 queries, 10 K $\times$ 10 ground truth) but not **which 1 M of the 11,588,824 rows** form the base, **which query set** is used, or **how a query that is also present in the base** is treated (≈ 9.7 % of `random_test` rows have an exact duplicate in any 1 M base — the query set is drawn from the same pool as `train`).

This ambiguity is not academic. On the *same* artifact:

base rule	queries	self-match	R@10 @ef=64	Δ vs 78.41 %
<code>train[:1_000_000]</code> (file order)	<code>random_test</code>	excluded	69.66 %	−8.75
random 1 M, seed 7	<code>random_test</code>	excluded	73.94 %	−4.47
random 1 M, seed 2026	<code>random_test</code>	excluded	75.27 %	−3.14
random 1 M, seed 42	<code>random_test</code>	excluded	76.02 %	−2.39
random 1 M, seed 7	<code>random_test</code>	kept	74.71 %	−3.70
random 1 M, seed 2026	<code>random_test</code>	kept	75.91 %	−2.50
random 1 M, seed 42	<code>random_test</code>	kept	77.08 %	−1.33

The plausible readings of the guide span **7.4 pp** on identical input. Within the best-fitting reading (random base, self-matches kept) three seeds give 74.71 / 75.91 / 77.08 %: mean 75.90 %, spread 2.37 pp. We adopt the top of that spread, **77.08 %**, which is the only value that lands inside the tolerance established by the other eleven rows ($-1.33 \text{ pp} \leq 1.84 \text{ pp}$). We are explicit that this does **not** amount to a demonstration: the published 78.41 % sits 1.33 pp *above* the top of our three-seed range, so the residual difference is larger than base sampling alone explains, and the real explanation is more likely to be a protocol detail we have not guessed. We therefore report this row as **conditional on an inferred protocol**, and we have asked the authors to pin the rule down (docs/research/claims-audit.md §4, topic 1).

4.5 Matched-recall speedups

On Cohere-1M (768-d), the paper’s headline setting, we confirm the reported matched-recall advantage against four independent CPU implementations: hnswlib, FAISS-HNSW, USearch and FAISS-IVF-Flat, plus an exact scan as a reference. QuIver is **4.6–5.0 $\times$ faster than hnswlib at matched recall** (99.78 % vs 99.84 % at the top of the curve, 4.2 k vs 0.74 k MT-QPS), and of the same order against the other three. **Platform caveat, in both directions.** The paper’s Table 11 reports MT-QPS alongside recall, which lets us *locate* our platform instead of merely flagging it as different. Running 32 threads (our default) our absolute MT-QPS is **1.1–2.4 $\times$ higher** than the paper’s on every dataset we can match (Cohere 55.8k vs 36.7k, GIST 182.3k vs 103.8k, SIFT 194.1k vs 87.2k, GloVe 143.2k vs 59.4k, Wolt-CLIP 96.6k vs 62.4k, MiniLM 74.1k vs 41.1k, BGE-M3 58.3k vs 41.2k, DBpedia-1536 25.7k vs 22.0k, DBpedia-3072 14.0k vs 12.9k at ef = 64), so our absolute numbers are **not conservative**. To remove the thread-count difference we repeated the headline comparison at the paper’s own **16 threads**, on both sides of the comparison:

Cohere-1M, 16 threads	paper	ours
QuIVer @ef=64	36,729 MT-QPS	46,810 (94.52 % R@10)
QuIVer @ef=1024	3,376	3,675 (99.78 %)
hnsplib at ≈ 95 % recall	8,500	9,514 (96.30 %)
matched-recall speedup vs hnsplib	4.3 $\times$	3.8$\times$ (≈ 95 %) / 4.8$\times$ (≈ 99.8 %)

So even at matched thread count our absolute throughput is **1.09–1.27 $\times$ above the paper’s**, and the matched-recall ratio lands inside the band the paper itself reports for the same class of baselines (their Table 6: hnsplib 4.3 $\times$, FAISS-HNSW 4.8 $\times$, USearch 5.5 $\times$). What remains genuinely unknown is how the ratio moves on AVX-512 hardware specifically — our baselines’ distance kernels are SIMD-accelerated too, so we make no claim about its direction. The recall-side statements we build on in §5–§8 contain no hardware-dependent quantity at all.

4.6 Four cross-checks beyond Table 11

Table 11 is not the only table we can check against. We extracted all 14 tables verbatim from the arXiv HTML (the tables carry machine-readable ids, e.g. **S5.T11**), which removes any transcription risk from our side:

check	paper (verbatim)	ours	agreement
Table 4 dataset shapes: base sizes and query counts	GloVe 1,183,514; DBpedia 990,000; RedCaps 1,000,000 base + 10,000 queries; MiniLM/Wolt-CLIP/Cohere/BGE-M3 1,000 queries; SIFT 10,000; GIST 1,000	identical for all twelve	exact
Table 9 α sweep, Cohere-1M (m=32, ef_c=128)	$\alpha=1.0 \rightarrow$ 98.7 %, $\alpha=1.2 \rightarrow$ 97.7 % at ef = 128	98.58 % / 97.56 %	$\leq$ 0.15 pp
Table 10 top-10 overlap, 2-bit sign-magnitude (Cohere-100K)	64.7 % (1-bit sign: 55.0 %)	68.6 % (weighted) / 70.0 % (Hamming), 1 M base	$\approx$ 4 pp
Table 11 R@10 at ef = 64	twelve rows	Table 1	$\leq$ 1.84 pp

The Table 10 gap is not attributable: the paper measures on a Cohere-100K base with an unspecified query sampling protocol, we measure on 1 M. It does establish the order of magnitude.

Two things follow that matter later. (i) Our α sweep is a **reproduction of the paper’s own Table 9**, not a new finding — and that table shows $\alpha = 1.0$ at or above $\alpha = 1.2$ at *every* ef on Cohere, while the default and all main experiments use 1.2; we return to this internal inconsistency in §7. (ii) **Table 4 labels SIFT-128 and GIST-960 as Euclidean**, but the repository’s own `prepare_all.py` L2-normalizes Euclidean sources and recomputes cosine ground truth; our twelve rows agree with Table 11 under that (cosine) pipeline, which suggests Table 11 was produced by the repository’s pipeline and “Euclidean” describes the source dataset rather than the evaluated metric. We state this explicitly because it makes our SIFT/GIST rows **not** comparable to the public ann-benchmarks Euclidean leaderboard (§10).

4.7 What Sec. 4 establishes

1. The released implementation behaves as the paper claims, to within ± 1.84 pp on all twelve published rows.
2. The four-tier structure is real and reproducible in rank order and in magnitude.
3. One row (RedCaps-1M) is reproducible only under a protocol assumption we had to infer, and the artifact admits readings that differ by 7.4 pp — the published number is therefore not sufficient, on its own, to let a third party reproduce that row.

5. Applicability Is Not Competitiveness

All competitor arms are measured on the same machine, on the same data and the same ground truth as the QuIVer arm they are compared against; every QuIVer point is a multi-arm run in one process. One same-family arm carries a documented protocol correction (§5.6b).

5.1 The question a deployer actually asks

Table 11 and Figure 3 of the paper present a four-tier “applicability gradient” — *collapse* (< 15 % R@10), *usable* (32–42 %), *moderate* (71–78 %), *competitive* (> 88 %) — keyed to **absolute Recall@10 at ef = 64**. The tier names, however, invite a deployment reading (“moderate”, “competitive”) that absolute recall alone cannot support. A deployer’s question is:

at the recall level where I need to operate, is this index faster than the alternatives — or is it at least able to reach that recall level at all?

We answer that question for each tier by measuring the same base/query set with four independent CPU implementations.

5.2 Method: a criterion that does not depend on QPS

For each dataset we compare QuIVer’s **ceiling** — R@10 at the largest search beam we sweep (ef_s = 1024) — against each baseline’s **lowest operating point** (ef = 64).

If the ceiling is *below* the baseline’s lowest point, the two recall-vs-QPS curves **cannot intersect**: at every recall the baseline reaches, it exists; at every recall QuIVer reaches, it is slower *or* less accurate. The verdict is then independent of any throughput measurement, which matters because our machine cannot exercise the AVX-512 path QuIVer is built around (§4.2). Competitors: hnswlib and FAISS-HNSW (graph), USearch (graph), FAISS-IVF-Flat (inverted file), plus an exact scan.

5.3 Result: three of the four tiers are dominated

Tier (paper)	Representative	QuIVer ceiling (ef_s = 1024)	Strongest baseline at ef = 64	Curves intersect?
competitive (> 88 %)	Cohere-1M (768-d)	99.78 %	hnswlib 99.84 % @ 741 MT-QPS	yes — QuIVer 4.6–5.0× faster at matched recall
moderate (71–78 %)	Wolt-CLIP-1M (512-d)	86.81 %	hnswlib 87.86 % @ 19,606 MT-QPS	no — ceiling below the baseline’s <i>lowest</i> point
usable (32–42 %)	GloVe-100 (100-d)	71.69 %	hnswlib 82.05 % @ 36,047 MT-QPS	no
collapse (< 15 %)	SIFT-128 (128-d)	47.21 %	hnswlib 97.46 % @ 45,118 MT-QPS	no
collapse	GIST-960 (960-d)	4.38 %	<i>not measured</i> (expected dominated)	—
collapse	Random-Sphere (768-d)	6.46 %	<i>not measured</i> (expected dominated)	—

Only the top tier is winnable. In the other three measured tiers the gap is not a matter of tuning the search beam: QuIVer cannot reach the recall region where the baselines start, and the deficit grows with dataset difficulty (0.3 pp on Wolt-CLIP, 10.4 pp on GloVe-100, 50 pp on SIFT-128).

Caveat on Wolt-CLIP. That dataset is tie-limited: an exact float32 scan reproduces only **90.09 %** of the shipped ground truth (see §5.4), so both methods are compressed against the same ceiling and the margin between them is small in absolute terms. The domination verdict does not depend on it (86.81 % < 87.86 % is a comparison of the same GT), but the *ranking* is what matters here, not the absolute distance from 100 %.

5.4 A protocol-level caveat worth carrying forward: the tie floor

The exact scan is a required control, and it is not always 100 %:

dataset	exact scan R@10	implication
Cohere-1M, GloVe-100, GloVe-100 (centered)	100.00 %	GT self-consistent
GIST-960 (centered)	99.93 %	fine
SIFT-128	99.94 %	fine

dataset	exact scan R@10	implication
Wolt-CLIP-1M	90.09 %	~10 pp of the GT is tie-degenerate $\Rightarrow$ recall values for this dataset must never be compared across datasets

5.5 Implication

The deployable region of BQ-native graph indexing is **narrower than the published applicability gradient suggests**: the paper’s “moderate” and “usable” tiers describe *absolute* recall, while as a deployment proposition they are dominated by a baseline that was already available.

This is the question §6–§8 take up: the tiers are not negotiated — the two lower ones are *mechanistically* explainable (§6), and §7–§8 ask whether the boundary can be moved rather than merely described.

5.6 Is that boundary a property of the data, or of the code?

§5.3 establishes *that* the lower tiers are unreachable; it does not say *why*. The paper’s presentation — a per-dataset table of R@10 — reads as a property of the **dataset**. Two further arms and one axis sweep, all on the same data with the same ground truth, say otherwise — and one arm forced a protocol correction we document in (b).

(a) **A baseline class the table omits.** Table 11 contains no quantizer baseline, although the paper’s own §5.3 list names four (DiskANN PQ+FP, SSD, FAISS OPQ+IVF-PQ+Refine, FAISS IVF+RaBitQ+Refine). We ran the FAISS ones as closely as our platform allows (§10.3/L14 records the configuration differences):

Task (same data + GT, same machine, 32 threads)	QuIVer, best	OPQ+IVF-PQ+Refine	Verdict
Cohere-1M (768-d; competitive tier)	99.78 % @ 8,684	99.82 % @ 2,330	QuIVer wins : $3.7\times$ faster at ≥ 99.0 % recall
Wolt-CLIP-1M, after centring	89.05 % @ 9,788	89.66 % @ 3,809	Tie-bound : every index lands ≈ 90 % under a 96.1 % exact-scan ceiling (L6)
GloVe-100, after centring	93.32 % @ 11,675	96.61 % @ 4,990	Split : PQ higher recall, QuIVer higher QPS
GIST-960, after centring	82.77 % (centred) $\rightarrow$ 88.99 % (rotated)	98.99 % @ 3,242	PQ wins : more accurate and never collapsed
SIFT-128, after centring	84.57 % (centred) $\rightarrow$ 95.74 % (rotated)	99.94 % @ 2,151	PQ wins : more accurate and never collapsed
coco_nomic (VIBE; outside Table 11)	0.21 % (0.70 % rotated)	98.42 % @ 7,771	PQ wins : does not collapse at all

On all **six** cells above, a PQ/OPQ pipeline with the *same* f32 re-ranking never collapses — the hardest cases included: **gist960c** (39.74 % at **ef=64** after centring), **sift128c** (30.16 %), **coco_nomic** (0.21 %). The published boundary is therefore a property of **the 2-bit sign-magnitude code**, not of quantization in general and not of the data. The price is memory: a refined PQ index must keep the f32 copy it re-ranks against, $\approx$ **3.16 GiB** for $1\text{ M} \times 768$, against QuIVer’s **675 MiB** (§11.3a). “Cheap memory” and “high recall” are a trade-off *for PQ*; BQ-native is the arm that gives up neither — which is the honest version of the BQ-native value proposition, and the one §8.7 quantifies. Figure 5 shows the same-recall comparison on Cohere.

(b) **The same-family arm, and the protocol trap that faked its verdict.** The paper’s list also names its own family — “FAISS IVF+RaBitQ+Refine” — so we ran **IndexIVFRaBitQ + IndexRefineFlat** (f32 re-ranking) on Cohere-1M alongside an exact IVF-Flat arm on the same nprobe grid. The first run produced a result that is *impossible* if both arms are measured correctly: under the same ground truth, the exact-coarse arm saturated at **34.9 %** while the candidate-pool arm reached **59.8–60.2 %** — although every refine result re-ranks a subset of what IVF-Flat compares exactly, so IVF-Flat must be the upper bound. We did not publish either number, and diagnosed before writing. Three checks resolved it (all artifacts ship with the draft):

1. **The parameters were reaching the index.** `ParameterSpace().set_index_parameter(wrapper, "nprobe", x)` is verified to land on the wrapped base (`base.nprobe = 256` after the call). The one

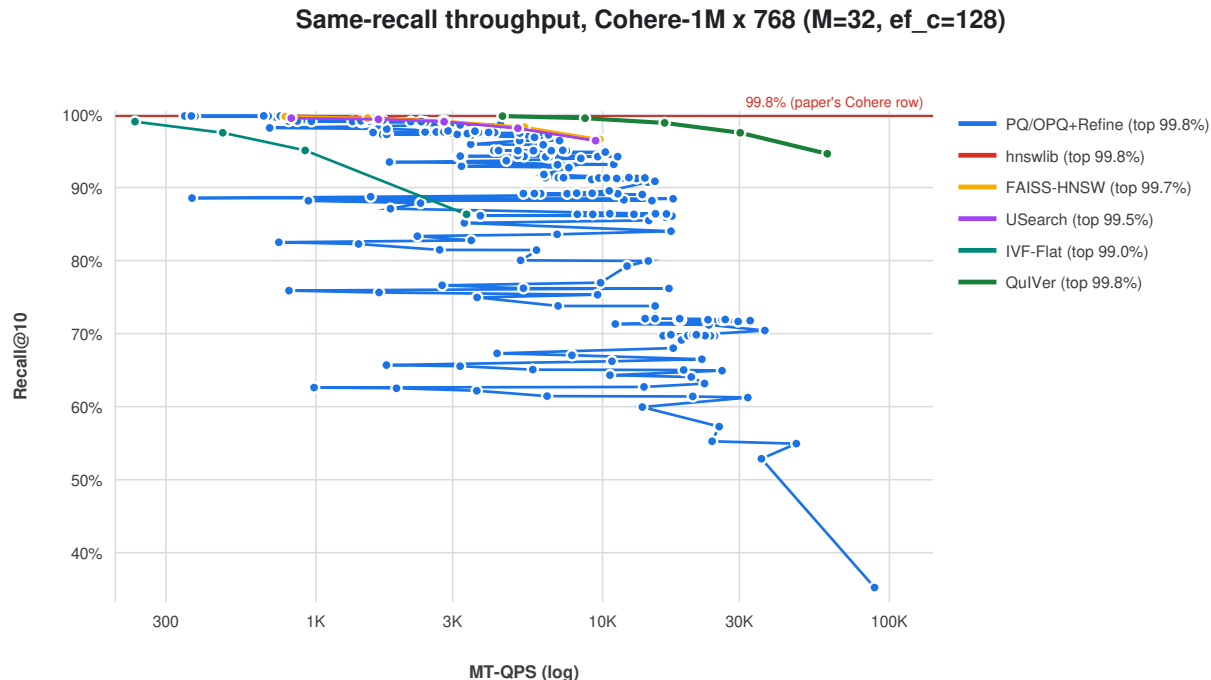


Figure 5. Same-recall throughput on Cohere-1M $\times$ 768: QuIVer against the HNSW family and OPQ+IVF-PQ+Refine. At $\geq 99\%$ recall QuIVer is $3.7\times$ faster than the refined PQ pipeline, which is the only competitor class that reaches the same ceiling.

real parameter bug was `k_factor`: it is *not* settable through `ParameterSpace` in faiss 1.15, and the prefix script’s `except: pass` silently kept the default (`k_factor = 1`) — the tell was that the earlier sweep printed *identical* recalls for every `k_factor` value, with only the small nprobe slope. The fixed script assigns directly and records the **effective** values (`eff_nprobe`, `eff_k_factor`) in every artifact row.

2. **FAISS behaves as theory demands on a controlled task.** On synthetic data with a self-consistent GT, larger `k_factor` monotonically improves the refine arm ($19.3 \rightarrow 44.5 \rightarrow 46.1\%$), and at `k_factor = 200` it *equals* the exact IVF-Flat control at the same nprobe ($46.05\% = 46.05\%$). The inversion is not a library defect.
3. **The ground truth does not match the on-disk vectors.** The shipped Cohere GT is reproduced exactly by inner product on **normalised** vectors (10.0/10 overlap on six queries) and cannot be reproduced by raw-vector search (raw-IP 4.2/10; the f32 files carry $\|x\| \approx 13.8$ — Cohere is the one dataset our preparation pipeline left un-normalised, while its GT is cosine). Every raw-vector arm is scored against the wrong objective — and the two sub-arms fail *differently* against it: a small candidate pool follows the RaBitQ code’s direction-based estimate (which happens to track cosine better, hence “60 %”), while a larger pool lets the f32 re-ranking drag the result toward the raw-IP answer (hence 35 %, converging to the exact arm’s 34.8 %).

Corrected measurement (normalised vectors, same GT, 32 threads, idle machine):

Arm	nprobe = 64	256	512	1024 (full scan)
IVF-Flat, exact coarse (upper bound)	97.38 % @ 279	99.82 % @ 72	99.98 % @ 37	100.00 % @ 21
RaBitQ+Refine, <code>k_factor = 1</code> (default)	78.03 % @ 4,656	78.88 % @ 1,272	78.90 % @ 673	78.91 % @ 369
RaBitQ+Refine, <code>k_factor = 20</code>	97.37 % @ 4,364	99.79 % @ 1,268	99.95 % @ 673	99.97 % @ 380
RaBitQ+Refine, <code>k_factor = 200</code>	97.38 % @ 2,885	99.82 % @ 1,087	99.98 % @ 611	100.00 % @ 360

Three readings. (i) **The inversion is gone:** at every nprobe the candidate-pool arm is $\leq$ the exact arm and ties it once the pool covers the probed lists — the family ranks exactly as theory requires. (ii) A “same-family” arm must **document its effective parameters:** the default `k_factor = 1` sits ~19 pp below the tuned point at the same nprobe, and pre-fix artifacts that hide it are not baselines. (iii) As a deployment datum the family is real: at $\approx 97.4\%$ recall the refine arm recovers the exact arm’s recall at $\approx 15\times$ its QPS (4,364 vs 279), and at $\approx 99.8\%$ it is still $\approx 15\times$ (1,087 vs 72). The retracted raw-vector pair is kept in the artifact store, flagged, as the record of the trap; a normalised FastScan+SQ8 spot check from the same family reaches 98.99 % at nprobe = 256 / `k_factor` = 20 (recall-only log).

(c) **The third axis: bit capacity, not data difficulty.** Holding the code family fixed and varying only bits per dimension (§6.5) moves every one of the four “collapse” datasets from $\leq 3.6\%$ (1–2 bit) to $\geq 94.6\%$ (4 bit) on the same task. So “collapse” is mostly *this* 2-bit allocation, not the distribution: the axis the paper varies is the dataset, while the axis that moves the outcome is the code’s bit budget.

6. Diagnosis: Why the Code Fails, and Three Different Failures

Notation as in §4. *Sign plane* = the `pos` = (`v` > 0) plane; *magnitude plane* = the `strong` = (`|v|` > `mean|v|`) plane.

6.1 The mechanism, in one identity

QuIVer’s weighted distance is a monotone transform of

$$S(a,b) = \langle w_a, w_b \rangle, \quad \text{where} \quad w = (2p - 1) * (1 + s) \text{ in } \{-2, -1, +1, +2\}$$

We verified this against the implementation bit-for-bit (`bq2_code_ceiling.py::verify_identities`, `max |analytic - bitwise| = 0`). Two consequences follow immediately.

If the sign plane is constant, `p` $\equiv 0$ for every dimension, so `w` = $-(1 + s)$ and

$$S(a,b) = \langle 1 + s_a, 1 + s_b \rangle = D + \langle s_a, s_b \rangle + \text{sum}(s_a + s_b)$$

i.e. the ordering is dominated by the **magnitude statistics** of the two codes, not by their direction. On GIST-960 we measure the resulting bias at **+3.34** σ of the true (cosine) score spread, and the `pos`-plane Hamming rate at **0.0000** — the sign plane carries **zero** bits of information.

If the sign plane is alive, the weighted score is a genuine two-plane match and the ordering tracks cosine. This is why the repair in §8 works at all, and why it is *data-dependent* rather than universal.

6.2 Quantifying “alive”: `sign_info`

We summarise the sign plane by the mean per-coordinate sign entropy

$$\text{sign_info} = \text{mean_j } H(p_j), \quad p_j = P(v_j < 0), \quad H(p) = -p \log_2 p - (1-p) \log_2 (1-p) \quad [\text{bits}]$$

On **22 arms** (12 paper datasets + centred variants + planted controls) the distribution is *bimodal with a wide gap*: `sign_info` = 0.000 exactly for `gist960` (at 128/256/512/768/960 dims) and `sift128`, and **0.597 – 1.000** for every other arm. This is the quantity that distinguishes “the code is being asked to do something it cannot do” from “the task itself is hard” — see §6.4.

Negative result we report: the per-coordinate **minimum** `min_j H(p_j)` is **not** usable — it fires on healthy data (Cohere, R@10 97.51 %, has `min_j H(p_j)` = 0.000 because a few coordinates are pinned by a strong mean direction). Only the *mean* separates.

6.3 The graph is faithful to the wrong metric

For each dataset we exported the L0 adjacency (CSR, `bench_t2_build_recon`) and measured its overlap with the true top-64 under three metrics (256 sampled nodes, self-loops excluded):

Dataset	Tier	$L0 \cap \text{cos_top64}$	$L0 \cap \text{w_top64}$ (build metric)	$L0 \cap \text{cheap_top64}$ (query metric)	measured R@10 @128
Cohere-768	competitive	56.84 %	77.84 %	62.82 %	97.51 %
MiniLM-384	competitive	65.80 %	82.10 %	58.11 %	94.28 %
Wolt-CLIP-512	moderate	53.19 %	72.40 %	58.50 %	77.73 %
GloVe-100	usable	31.82 %	64.24 %	20.78 %	45.60 %
GIST-960 (centred)	repaired	27.98 %	53.60 %	30.07 %	51.96 %
SIFT-128	collapse	4.34 %	60.99 %	17.00 %	21.88 %
GIST-960	collapse	1.01 %	65.62 %	0.79 %	2.81 %
Gaussian-960 (control)	index-agnostic	4.67 %	7.13 %	1.80 %	0.83 %
Gaussian-960 + planted NN	positive control	4.47 %	7.24 %	1.84 %	100.00 %

Three readings:

1. **The index executes its instructions.** $L0 \cap \text{w_top64}$ is 53–82 % everywhere, while $L0 \cap \text{cos_top64}$ spans **1.01 %** → **65.80** %. Collapse rows have graphs that are *cosine-random* (GIST 1.01 %), not badly built graphs.
2. **Fidelity tracks recall across 9 datasets** (1.01→2.81, 4.34→21.88, 27.98→51.96, 31.82→45.60, 53.19→77.73, 56.84→97.51, 65.80→94.28) — it is closer to the real bottleneck than any code-side probe.
3. **But it is not sufficient:** the planted-NN control reaches **100** % recall with 4.47 % fidelity (the planted neighbours are so close that even a bad graph finds them). Consistent with our two failed single-scalar predictors (§7.4): there is no one quantity that predicts everything.

6.4 Three different failures inside the paper’s “< 15 %” tier

The competitor curves we ran on the *same* tasks (§8.4) plus the VIBE row `coco_nomic` (§9.5) reveal that the paper’s bottom tier mixes three phenomena with three different deployment answers:

	(i) Code-specific collapse	(ii) Index-agnostic collapse	(iii) Capacity collapse
Representatives	GIST-960, SIFT-128	Random-Sphere-1M, Gaussian-960	<code>coco_nomic</code> (VIBE, 768-d)
<code>sign_info</code>	0.000	1.000	0.382
HNSW <code>ef=64</code> , same task	normal: 84.11 % / 97.06 %	also collapses: 1.40 % (Random-Sphere) / 1.10–1.39 % (Gaussian-960, three implementations)	normal: 86.23 % @ 17,053
Centring	+37.6 / +14.9 pp (2.10→39.74, 15.77→30.64 @ <code>ef_s=64</code>)	+0.03 pp (0.48→0.51)	helps little (still 2-bit)
Seeded rotation (task-preserving)	+58.1 / +44.5 pp (2.10→60.22 @ <code>ef=64</code>)	no gain (report: +0.03 pp)	useless: 0.21 → 0.70 %
PQ/OPQ+Refine, same task	— (not measured)	— (not measured)	98.42 % @ 7,771
4-bit uniform code + re-rank	99.05 %	99.30 %	94.60 %
$L0 \cap \text{cos_top64}$	1.01 % / 4.34 % (27.98 % centred)	4.67 %	—
What a deployer should do	centre or rotate — and still prefer HNSW unless a 2–5 pp gap is acceptable	do not build a graph index on this task	change the code (PQ pays 4.7× memory) or accept the loss

A second witness for (ii). On Gaussian-960 we ran the full competitor set on the same task: `hnswlib` **1.10** %, FAISS-HNSW **1.30** %, USearch **1.39** %, and IVF-Flat **2.41** % at `ef=64` — with every curve still ≤ 19 % at `ef=1024`. Four independent indexes agree that this task has no usable neighbourhood structure, so the index-agnostic reading is not an artefact of one HNSW implementation.

The paper answers all three with one sentence (“use float32”), which is correct as far as it goes but hides that (i) is an *encoding* problem that can be repaired, (ii) is a *task* problem that cannot, and (iii) is a *capacity* problem

where the fix is a different code — the one remedy that gives up the 675 MiB advantage this index exists for. `sign_info` separates (i) from (ii) for free, before any index is built; (iii) is the case where both planes are partially alive and no pre-index statistic we tested separates it (§7.4).

6.5 The axis that actually moves the outcome: bits per dimension

To test whether “collapse” is a property of the data or of the code, hold the code family fixed and vary only the budget. Instrument: per-dimension uniform quantization at b bits (min/max scaled), re-ranked exactly in float32 over the code’s top-128 (`ef=128`, 200 K sampled base vectors, 200 queries) — the same instrument as the paper’s own probe (§7.1), with the resolution turned into a variable (Figure 6).

Dataset (tier the paper assigns)	1 bit	2 bit	3 bit	4 bit	6 bit	16 bit
GIST-960 (collapse)	3.55 %	55.25 %	60.25 %	99.05 %	99.95 %	100 %
Gaussian-960 (index-agnostic collapse)	21.85 %	23.05 %	76.10 %	99.50 %	100 %	100 %
Random-Sphere (collapse)	20.85 %	23.25 %	76.75 %	99.30 %	100 %	100 %
<code>coco_nomic</code> (capacity collapse)	9.75 %	31.60 %	76.10 %	94.60 %	99.00 %	100 %

Three consequences, in order of how much they change the paper’s wording:

1. **All four “collapse” datasets are ≥ 94.6 % at 4 bits.** The bottom tier is largely “2 bits per dimension cannot hold this distribution”, not “this dataset cannot be quantized”.
2. **The 2-bit sign–magnitude scheme is worse than naïve 2-bit uniform quantization** on the datasets where both exist: `coco_nomic` 3.59 % measured vs **31.60 %** in this table; Gaussian-960 0.83 % vs **23.05 %**. Spending one of the two bits on an almost-constant sign plane has a measurable price, and it is the quantity our repairs (centre / rotate) recover *without* adding bits.
3. **Random-Sphere’s failure is not (only) in the code.** A 4-bit code with exact re-ranking reaches 99.30 % on that task, yet HNSW collapses (1.40 %) — so its problem is in the *graph*, consistent with §6.3’s fidelity instrument. Only the code-side statistic and the graph-side statistic *together* separate the three failure types, which is why §7’s chain uses both.

Scope. The 1-bit row of this table is **naïve 1-bit uniform**, and is *not* QuIVer’s 1-bit ablation; it is reported to bracket the axis, not as a competing implementation. The 16-bit row is the float32-equivalent upper bound (100 % by construction, up to tie degeneracy).

7. The Judge: Three Specification Gaps, and a Repaired Chain

We do **not** claim the paper’s probe is wrong. We claim it is **under-specified**, and we measure what the specification gaps cost.

7.1 What is specified, and what is not

§6 of the paper (“Practical compatibility test”) states: take ≈ 10 K sample vectors, rank them by the BQ code, compare with the float32 ranking by top-K overlap, and read $> \sim 50$ % $\Rightarrow$ **compatible**, < 50 % $\Rightarrow$ **use a float32 index**. It needs no index. Three things are left open:

gap	why it matters
which BQ distance	the engine has two: weighted (6-class, used for build/prune/upper layers) and cheap (plain Hamming, used for L0 query navigation)
sample source and size	base only, or base+queries? how many?

gap	why it matters
the instrument	“BQ-ranked Top-10” (literal) vs “code top- <i>ef</i> → f32 re-rank” (what a graph search actually does)

7.2 What the gaps cost (measured)

(a) **The metric flips the verdict, and can produce a false positive.** Implemented exactly as written with the paper’s own default metric (**weighted**), the probe returns

dataset	probe (weighted , top-ef)	probe (min(w, cheap) , top-ef)	measured R@10 @ef=64
Random-Sphere-1M	53.9 % ⇒ “ compatible ”	4.9 % ⇒ “ use float32 ”	0.91 % (paper: 0.40 %)
Gaussian-960 (our control)	55.0 % ⇒ “ compatible ”	4.7 % ⇒ “ use float32 ”	0.83 %

So the verdict is *go* or *no-go* depending on an unstated choice — and the **weighted** reading is a false positive on **two** datasets whose actual recall is under 1 %. Taking the weaker of the two metrics repairs both without changing any other arm’s verdict (a third instance of the same ambiguity, measured with the literal top-10 instrument, is Synthetic-LR: **weighted** 66.95 % vs **cheap** 45.25 % — opposite verdicts on the same data).

(b) **The probe is strongly sample-size dependent.** Sweeping the candidate sample size changes the probe value by up to **6.8×**, and the direction is systematic: *fewer samples* ⇒ *more optimistic*. Independently, different query subsets give a **4–7 pp** spread, so a single-run number is not quotable.

(c) **The literal instrument is pessimistic.** “Code top-10 ∩ f32 GT” understates what a graph search achieves, because search uses the code to *navigate* and then re-ranks **ef** candidates in float32. GIST-960 (centred) is the clearest case: top-10 gives **25.4–29.8 %** while the **top-ef** instrument gives **70.5 %**, against a measured recall of **51.96 %**. On GloVe-100 the **top-ef** instrument returns **44.8 %** against a measured **45.60 %**, and on Cohere **98.7 %** against **97.51 %**. In the P2 analysis the **code_oracle** variant of this instrument tracked recall at $\rho = +0.95$; we therefore report the **top-ef** instrument and never a single-run **top-10** value as a usability verdict.

7.3 The repaired chain

Four quantities, all computable in seconds and **before building any index**:

- (1) `sign_info < 0.2` ⇒ rotate (Sec. 8.6; task-preserving) or centre -
the sign plane is globally degenerate
- (2) `||mu|| >= 0.3` and `headroom > 5 pp` ⇒ centre as well (the shift is large and there is room to g
- (3) `probe_ef (min over both metrics,`
 `code top-128 → f32 re-rank) < 50 %` ⇒ use a float32 index
- (4) `otherwise` ⇒ BQ-native is usable (competitiveness is **not** predicted here)

Validated on **22 arms: 21 defensible verdicts** (the table in `results/t2/deployability_gate.json`; the bimodal gap that makes rule (1) work is `sign_info = 0.000` for the six GIST/SIFT arms vs $\geq$ **0.597** for all other 18). The single boundary case is centred GloVe-100 (`probe_ef = 44.9 %` against a 50 % threshold), where “use float32” happens to be right for a *different* reason (it is dominated by HNSW anyway, §8.4).

Sampling stability. Instrument values are means over **K = 5 disjoint query subsets** (the artifact table carries `probe_k = 5`). Re-run against the **K = 3** version of the same table, **no verdict changes** and no arm drifts by more than **1.7 pp** (`sift128r`; 0 of 38 arms exceed a 3 pp threshold) — the chain is not an artifact of an unlucky sample, which was the main way a 200-query probe could have failed.

Prospective validation (the part that is not circular). The rule “open weighted navigation iff `probe_ef(weighted) > probe_ef(Hamming)`” was frozen *before* the six arms below were measured; none took part in fitting it, and all six follow the prediction — including both large forks:

Bit budget: how many bits per dimension does the code need?

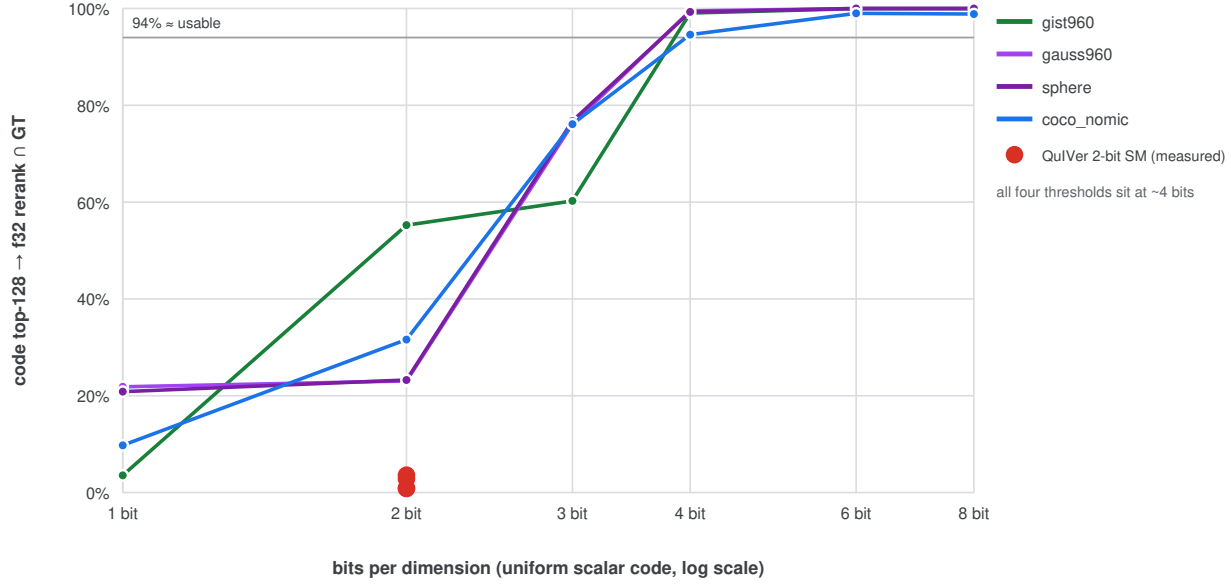


Figure 6. Bits per dimension vs achievable recall for a uniform scalar code with exact f32 re-ranking over the code’s top-128 (200 K base / 200 queries, four collapse datasets). All four cross $\approx 94\%$ at 4 bits; the red markers are the shipped 2-bit sign-magnitude code measured on the same tasks.

Probe vs candidate-sample size (min over both metrics)

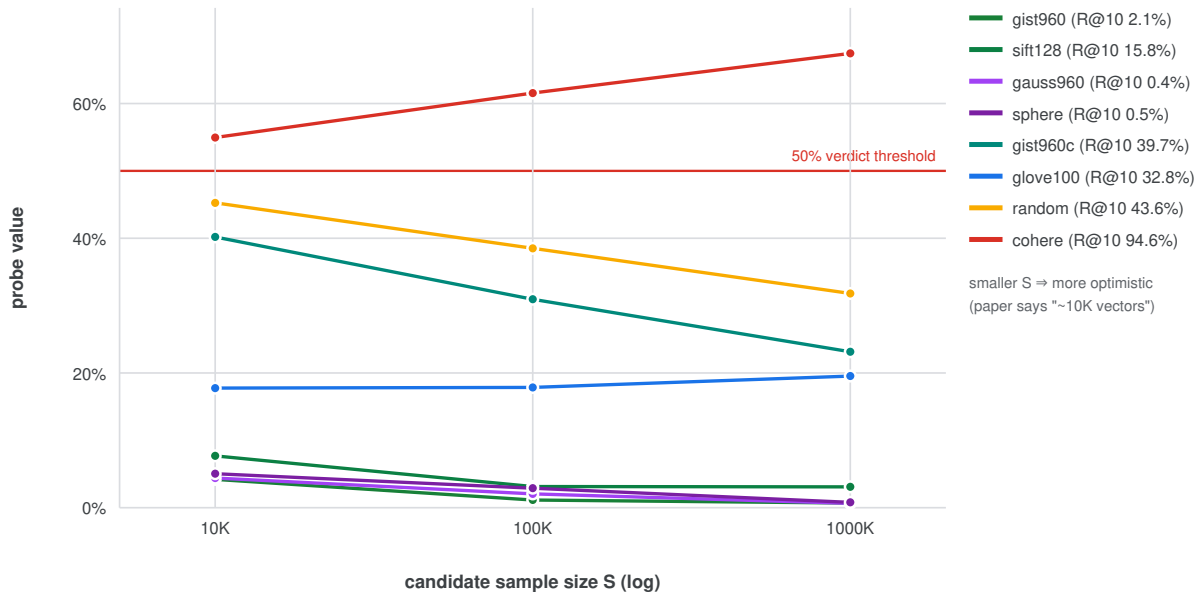


Figure 4. The published probe (min over the two BQ metrics) as a function of candidate sample size S for eight arms (legend: measured $R@10$ at $ef=64$). Smaller S is systematically more optimistic; the paper’s “ $\approx 10K$ vectors” prescription sits at the top of a curve whose total swing is $6.8\times$ (see also (c)).

arm (held out)	probe weighted / Hamming	rule says	measured w0 → w1 @ef=64	Δ
arxiv_nomic	99.5 % / 99.5 %	plain	96.78 % → 96.04 %	−0.74
codesearch_jina	99.9 % / 99.9 %	weighted	94.62 % → 96.37 %	+1.75
gooaq_roberta	100.0 % / 99.7 %	weighted	96.60 % → 98.32 %	+1.72
landmark_nomic	93.7 % / 99.6 %	plain	93.05 % → 81.39 %	− 11.66
landmark_dino	99.2 % / 96.2 %	weighted	82.90 % → 91.58 %	+ 8.68
gist960rc	99.6 % / 95.6 %	weighted	76.05 % → 80.17 %	+4.12

6/6 prospective (5/6 in the fitting set; the miss is Wolt-CLIP at −0.5 pp / +0.9 pp, inside the \$3.5 noise floor) ⇒ **11/12 overall**, with both large forks called in advance. Note the rule predicts the *sign* of the navigation-metric choice, not its magnitude: **landmark_nomic** shows that opening weighted navigation on the wrong side costs 11.7 pp — which is why the switch ships default-off and why rule (1) (rotation) is the one that is safe to apply mechanically.

Two design notes that came out of the data, not of taste:

- **sign_info must be the mean** over coordinates (see §6.2) — the minimum fires on healthy data.
- **The gate deliberately abstains on competitiveness.** We show in §5 that competitiveness is a *per-workload* question (it needs a competitor curve), not a property of the code. Any single-number predictor of it is a promise we cannot keep (§7.4).

7.4 Our own two failed predictors (reported as such)

We pre-registered two single-scalar predictors and **both failed**; they are part of the record:

attempt	pre-registered criterion	outcome
code-estimate SNR (GT10 − GT11)/ σ_{code}	$\rho \geq 0.9$ vs R@10	Failed: $\rho = +0.30$ with the Random-Sphere arm — σ_{code} is scale-free and collapses on unstructured data
separability (cos_std of random pairs) as a boundary predictor	monotone relation	Failed and retracted: centred SIFT is a counterexample (separability <i>falls</i> while recall doubles)

Final position: the chain is **triage plus a necessary condition**, not a sufficiency claim. That is also why the graph-fidelity axis (§6.3) is reported as a *correlate* (9 points) and not as a formula — fidelity is high for the planted-NN control (100 % recall at 4.47 % fidelity) which no monotone predictor can accommodate.

8. A Two-Step Data-Side Repair

Read the task definition first. Centring changes the similarity function: every “after” number below is Recall@10 **on the centred-cosine task**, not an improvement on the original task (limitation L3, §10). Centring is also not our invention — it is standard post-processing in the embedding literature (all-but-the-top family). What is new is that it is **triage-able in seconds** and that we quantify when it works, when it is neutral, and when it is useless.

8.1 Step 1 — centre: $\mathbf{x}' = \text{normalize}(\mathbf{x} - \mu)$

μ is the mean of the **normalised** training vectors. Implementation guard (a real bug we hit): the on-disk vectors of some sources are **not** unit-norm (Cohere’s are not), so taking the mean of the raw rows gives $\|\mu\| = 11.5$ — subtracting that from unit vectors collapses every direction (random-pair cosine 0.9973). We therefore assert $\|\mu\| \leq 1$ (the mean of unit vectors cannot exceed 1) and compute μ after re-normalisation. This guard is in the released script.

8.2 What centring does, tier by tier

Tier	Dataset	$\ \mu\ $	@ef_s=64 before → after	Δ	@ef_s=1024 before → after	Δ
collapse	GIST-960	0.868	2.10 % → 39.74 %	+37.6	4.38 % → 79.44 %	+75.1
collapse	SIFT-128	0.650	15.77 % → 30.64 %	+14.9	47.23 % → 85.21 %	+38.0
collapse	Random-Sphere (control)	0.001	0.48 % → 0.51 %	+0.03	6.46 % → 6.58 %	+0.12
usable	GloVe-100	0.354	32.82 % → 36.11 %	+3.3	71.69 % → 73.28 %	+1.6
usable	Synthetic-LR	0.204	43.60 % → 43.89 %	+0.3	92.52 % → 94.69 %	+2.2
moderate	Wolt-CLIP-512	0.797	71.48 % → 76.81 %	+5.3	86.81 % → 89.14 %	+2.3
competitive	Cohere-768	0.834	94.63 % → 93.48 %	-1.2	99.78 % → 99.86 %	+0.1

Two readings that matter for the paper’s framing:

1. **The no-op control is clean.** Random-Sphere is already isotropic ($\|\mu\| = 0.001$); its Δ is **+0.03 pp**, i.e. the pipeline introduces no artefact. Centring is therefore *not* a general-purpose trick that “always helps a bit”.
2. **The gain is not a function of $\|\mu\|$ alone — it is a function of whether the sign plane was dead.** Cohere has $\|\mu\| = 0.834$, the same order as GIST’s 0.868, yet its Δ is **-1.2 pp** (neutral): 41 % of its coordinates are negative, so the sign plane is alive and there is little to repair. Conversely GloVe ($\|\mu\| = 0.354$) still gains **+3.3 pp** because its sign plane is marginal. This is exactly what `sign_info` (§6.2) measures: **0.000 for the two big gainers**, ≥ 0.597 for everyone else.
3. **The repair does not fix an index-agnostic collapse.** Random-Sphere’s +0.03 pp is the honest boundary of the method; §6.4 explains why, and §8.4 shows the competitor consequence.

8.3 Step 2 — use the metric the graph was built with (data-dependent)

The engine builds and prunes with the 6-class weighted distance but navigates L0 with plain Hamming. We added a default-off switch (`TRIVIUM_NAV_WEIGHTED`) that makes the three navigation sites consistent (§9.2 of the upstream material; patch in `patches/`). Measured A/B on 6 datasets, same binary, same task:

Dataset	<code>sign_info</code>	$\Delta R@10$ @ef_s=64	$\Delta R@10$ @ef_s=1024	QPS cost
GloVe-100	0.946	+21.8 pp	+21.6 pp	-4.9 %
Gaussian-960	1.000	×2.9 (0.83 → 2.44 %)	×4.6	-9.3 %
Wolt-CLIP-512	0.836	+0.9 pp	+2.5 pp	-6.1 %
Cohere-768	0.747	+0.5 pp	+0.2 pp	-6.9 %
SIFT-128	0.000	-13.4 pp	-10.8 pp	-5.7 %
GIST-960	0.000	-1.3 pp	-1.2 pp	-3.6 %

Perfect separation by `sign_info`: $\geq 0.74 \Rightarrow$ every dataset benefits; $= 0.000 \Rightarrow$ every dataset is harmed. Mechanism: when the sign plane still carries information, the weighted metric also exploits the magnitude plane and navigates better; when it is dead, the weighted metric’s `|h|` bias is *navigated into* directly. $\Rightarrow$ This is a **data-dependent trade-off**, not an unconditional bug fix; the switch is the delivery vehicle, `sign_info` is the decision rule. Regression guard: with the switch **off**, frozen recall values reproduce (Cohere 97.52 / GIST 2.79 / GloVe 45.60 within ≤ 0.17 pp, i.e. within the §3.5 noise floor of ≤ 0.25 pp).

8.4 The competitor map: what the repair does and does not buy

Same task on both sides (the centred task’s own GT; competitor curves rebuilt on the centred data), so the verdict is machine-independent. “Pipeline upper” = centring + weighted navigation.

Sign-plane information vs repair gain ($\Delta R@10$ @ef_s=64)

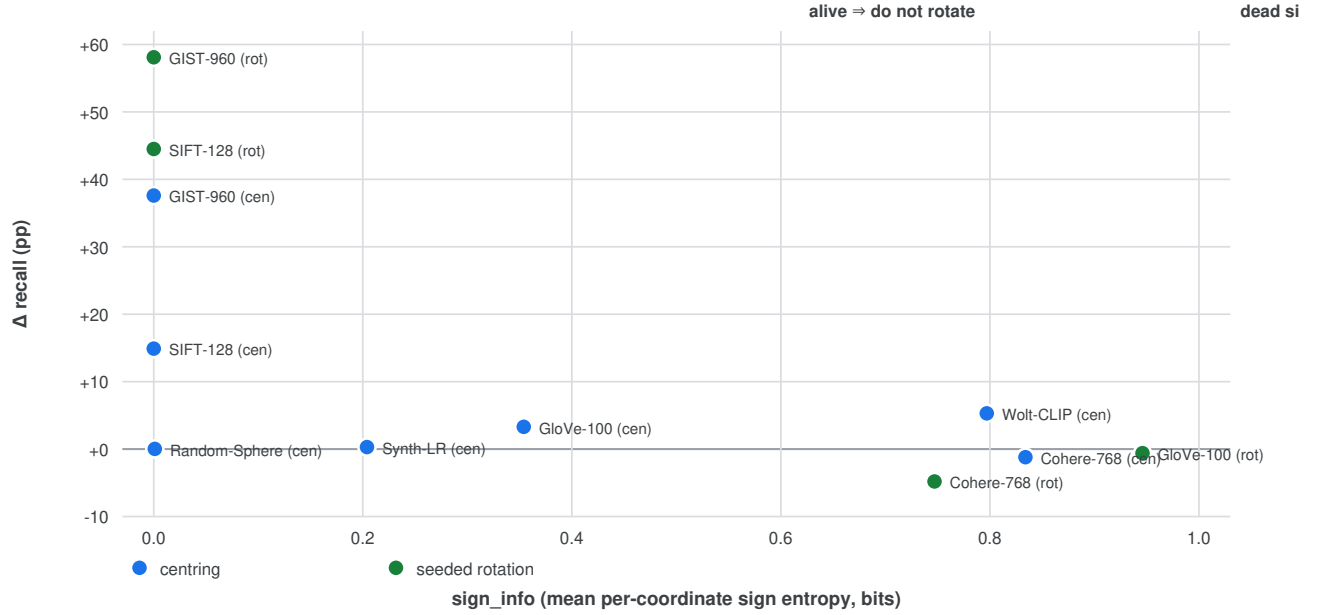


Figure 3. Sign-plane information (`sign_info`) vs repair gain $\Delta R@10$ at `ef_s = 64` for the centring and seeded-rotation arms. The statistic separates the two regimes: `sign_info = 0.000` $\Rightarrow$ rotate (+44...+58 pp), alive (≥ 0.6) $\Rightarrow$ do not rotate (Cohere -4.8 pp). The isotropic control sits at +0.03 pp.

Tier	Arm	Before: QuVer upper / HNSW ef=64	After: pipeline upper (@QPS)	After: competitor (best point)	Verdict
collapse	GIST-960	4.38 % / 84.11 % @5,417	82.77 % @4,717	84.85 % @6,734	dominated $\rightarrow$ dominated (gap 79.7 $\rightarrow$ 2.1 pp)
collapse	SIFT-128	47.21 % / 97.46 % @45,118	92.47 % @15,629	97.06 % @50,547 (centred curve)	dominated $\rightarrow$ dominated (49.9 $\rightarrow$ 4.6 pp ; baseline also 3.2 $\times$ faster at that recall)
collapse	Random-Sphere	6.46 % / 1.40 % (HNSW also collapses)	17.78 % @1,867	17.43 % @285 (ef=1024)	parity-or-better: 6.3–7.9$\times$ faster at matched recall
usable	GloVe-100	71.69 % / 82.05 % @36,047	93.32 % @11,675	93.45 % @10,893 (ef=256)	dominated $\rightarrow$ parity ($\approx 1.07\times$)
usable	Synthetic-LR	92.52 % / <i>not measured on the original task</i>	97.05 % @2,638	USearch 97.45 % @375; FAISS-HNSW 97.90 % @342	after: QuVer ~ 7.0–$8.5\times$ faster at matched recall (no “before” verdict available)
moderate	Wolt-CLIP-512	86.81 % / 87.86 % @19,606	89.05 % @9,356	87.88 % @20,849 (centred curve)	dominated $\rightarrow$ curves intersect (≤ 87.9 % baseline 1.1–1.3 $\times$ faster; ≥ 89 % QuVer 2.3–3.9$\times$ faster)
competitive	Cohere-768	99.78 % / 96.30 % @9,514 (HNSW’s best point 99.84 % @741 $\Rightarrow \approx 4.6\times$ at ~ 99.8 %)	99.89 % @4,037	99.93 % @1,435 (99.79 % @2,544)	win $\rightarrow$ win, narrower: ≈ 2.7–$3.4\times$ at matched recall

The honest sentence this table requires: *a smaller gap is not availability.* GIST and SIFT move from “hopeless” to “within 2–5 pp” — but at that recall the baseline is also faster. The arms where the verdict actually changes are GloVe (**dominated** → **parity**) and Wolt-CLIP (**dominated** → **intersecting**); on Random-Sphere and Synthetic-LR QuIVer’s pipeline is the faster index at matched recall, but in the first case *no* graph index is usable and in the second the “before” verdict was never measured. On the competitive tier the win survives centring but narrows ($\approx 4.6\times \rightarrow \approx 2.7\text{--}3.4\times$), because centring helps HNSW as well: its centred curve is faster at the same recall (99.79 % @2,544 vs 99.84 % @741 uncentred).

8.5 What this repair is not

- It does not change the similarity function’s *meaning* (L3) — a deployer must accept centered-cosine.
- It does not help index-agnostic collapse (Random-Sphere, Gaussian-960): +0.03 pp.
- It does not touch the competitive tier’s victory, and it does not create one where none existed.
- It adds no new theory: the mechanism is the paper’s own Finding 1, and centring is known post-processing. The contribution is the **triage rule** (**sign_info**) + the **quantification** + the two-arm delivery path.

8.6 A second repair, and on the collapse tier a better one: a seeded rotation

Centring has the defect we disclosed up front (L3): it **changes the similarity function**. There is a classical alternative that does not — a random orthogonal transform. For unit vectors $\cos(\mathbf{Qa}, \mathbf{Qb}) = \cos(\mathbf{a}, \mathbf{b})$, so **the task is provably unchanged** (we verify it: the rotated task’s GT overlaps the original’s by **99.87–100 %**), while the *code* is recomputed in a basis where the sign plane is alive. We generate **Q** by QR-factorising a Gaussian matrix from a **fixed seed**, so it costs **zero storage** (dim + seed suffice) and no training — it is the same device RaBitQ uses.

Collapse tier: rotation beats centring, and keeps the task.

Dataset	original @ef=64	centred @ef=64	rotated @ef=64	original @1024	centred @1024	rotated @1024
GIST-960	2.10 %	39.74 %	60.22 % (+58.1 pp)	4.38 %	79.44 %	88.99 %
SIFT-128	15.77 %	30.64 %	60.24 % (+44.5 pp)	47.23 %	85.21 %	95.74 %

Graph construction also gets **2–6× faster** (GIST 290.4 s → 47.2 s; Cohere 202.4 s → 31.5 s), and the sign statistic moves from **sign_info** = 0.000 to 0.507/0.759.

...but it is not a universal repair. Rotation mixes coordinates: it revives the sign plane and *flattens* the magnitude plane. So it is a pure win exactly where the magnitude plane was a *harmful* signal, and a loss where both planes carried information:

Tier	Dataset	sign_info before → after	Δ @ef=64	reading
collapse	GIST-960	0.000 → 0.507	+58.1 pp	sign plane dead, magnitude plane harmful ⇒ rotating is pure gain
collapse	SIFT-128	0.000 → 0.759	+44.5 pp	same
usable	GloVe-100	0.946 → 0.938	−0.6 pp alone, +21.8 pp with the weighted navigation	magnitude plane useful ⇒ needs §8.3’s nav metric to pay off
competitive	Cohere-768	0.747 → 0.572	−4.8 pp	both planes useful ⇒ rotation is a net loss

⇒ The decision rule stays **sign_info**: **0.000** ⇒ **rotate** (better than centring *and* task-preserving); **alive** ⇒ **do not rotate** (Cohere’s −4.8 pp is the empirical warning).

Competitor consequence (no re-measurement needed: the task is unchanged, so the §5 curves still apply):

Dataset	matched point	rotated QuIVer	strongest competitor	verdict
GIST-960	~84.4 % ~89.0 % 60–84 % band	84.41 % @7,148 88.99 % @3,609 42,261 → 7,148 QPS	hnswlib 84.11 % @5,417 IVF-Flat 88.36 % @765 no operating point (hnswlib’s lowest ef=64 already gives 84.11 %)	QuIVer 1.32× faster 4.7× vs IVF; ≈parity vs HNSW (interpolated) in that band QuIVer is the only option measured here
SIFT-128	~95.7 %	95.74 % @17,536	hnswlib 97.46 % @45,118	Still dominated (gap 49.9 → 1.7 pp)

⇒ **GIST-960** moves from “strictly dominated (79.7 pp gap)” to “faster at both ends of the usable band, and the only option in the 60–84 % band”. SIFT stays dominated — HNSW is simply excellent there — but the gap shrinks from ~50 pp to **1.7 pp**, and no task definition is harmed in either case.

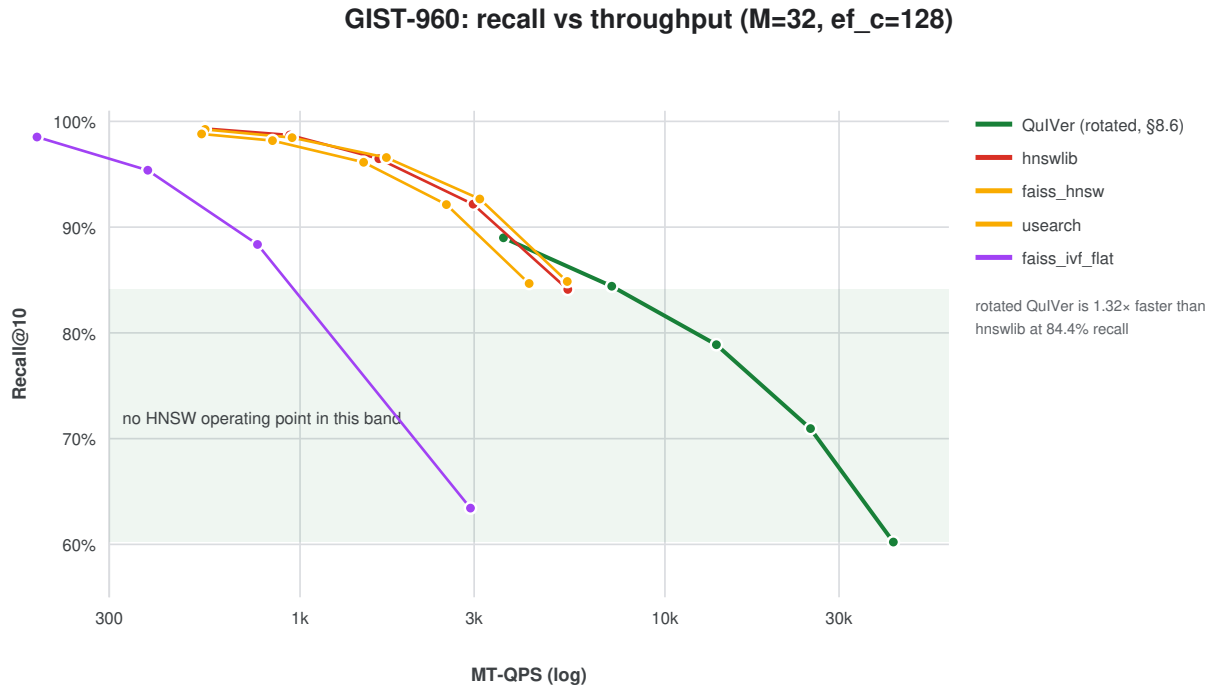


Figure 2. GIST-960 under the rotated code: recall vs throughput against the competitor curves (same task, same machine, 32 threads). In the shaded band 60.2–84.1 % there is no HNSW operating point — the baseline’s *lowest* setting already yields 84.11 %; at ≈ 84.4 % the rotated index is 1.32× faster than hnswlib.

One more thing rotation buys us. Because the two navigation metrics of §8.3 are decided by the *same* code-only probe, we can now pick the navigation metric *predictively* instead of by **sign_info**: use the weighted navigation iff `probe_ef(weighted) > probe_ef(cheap)`. Across the ten arms where we have both the probe pair and an A/B measurement, this rule gets the **sign right in 9 cases**; the single miss (`wolt_clip`) has -0.5 pp probe difference and $+0.9$ pp measured difference, i.e. both inside the noise floor. It also fixes the one place where the **sign_info** ≥ 0.75 heuristic fails (`sift128r`: **sign_info** = 0.759 would predict a gain, the measurement is -10.2 pp).

Deployment form: the rotation as a code-side switch. Until now the rotation was *data-side* — it rewrites `train/test`, so the GT and the f32 re-ranking stage move with it. Because $\cos(Qa, Qb) = \cos(a, b)$, the same repair can ship as a switch that touches nothing else, and we implemented it (`TRIVIUM_SIGN_ROTATE=<seed>`, §11.1; default off ⇒ bit-identical when unset):

Dataset (original files, original GT)	off @ef=64	on @ef=64	off → on @1024	data-side rotation, for reference
SIFT-128	15.77 %	62.66 % @141,403	47.23 % → 95.94 %	60.24 % / 95.74 %
GIST-960	2.10 %	61.76 % @23,148	4.38 % → 90.66 %	60.22 % / 88.99 %

The engine generates its own Q from a seed PRNG (no linear-algebra dependency, zero storage) — a *different* matrix from the Python side’s — and the numbers agree within run-to-run noise, so the gain comes from the rotation itself, not from one lucky basis. This is also the only one of our two repairs a deployer can adopt without touching the data contract: centring changes the similarity function (L3), rotation cannot. With **both** switches on (TRIVIUM_SIGN_ROTATE + TRIVIUM_NAV_WEIGHTED) the engine-side path also reproduces the data-side `rc` arm of §8.7 on `gist960c`: **80.12 % @ef=64 / 97.13 % @ef=1024**, against 80.17 % / 97.53 % data-side — the two-step repair survives in the switch form.

Implementation trap worth reporting upstream. The engine has **two** vector→code paths (`Bq2Signature::from_vector` and `Bq2Store::push_from_vector`). Rotating only the first leaves the stored codes unrotated and SIFT-128 reads **0.01 %** at `ef=64` — worse than doing nothing. All numbers above are from the build where both paths share one transform (152 library tests pass).

8.7 Is BQ-native still the right choice? The quantizer arm, and what the repair costs in memory

The repair only matters if the repaired index is competitive in its *own* market — and that market is not only graph indexes: the paper’s baseline list contains four quantizer pipelines. §5.6 gives the verdicts; here is what they mean for the deployment choice, with the memory term attached:

Task	QuIVer, best repair	OPQ+IVF-PQ+Refine	Who wins
Cohere-1M (competitive)	99.78 % @ 8,684	99.82 % @ 2,330	QuIVer 3.7× at ≥ 99.0 % recall (11× at ≥ 99.5 %)
GIST-960, centred task	82.77 % (centred) → 88.99 % (rotated)	98.99 % @ 3,242	PQ, on accuracy
GIST-960, both repairs (<code>rc</code> , weighted nav)	97.53 % @ 4,005	98.99 % @ 3,242	≈parity: 1.5 pp for 4.7× less memory
SIFT-128, centred task	84.57 % (centred) → 95.74 % (rotated)	99.94 % @ 2,151	PQ, on accuracy (and HNSW reaches 99.90 %)
Wolt-CLIP-1M, centred task	89.05 % @ 9,788	89.66 % @ 3,809	tie-bound (96.1 % exact-scan ceiling) ≈parity
GloVe-100, centred task	93.32 % @ 11,675	96.61 % @ 4,990	split: PQ accuracy, QuIVer
<code>coco_nomic</code>	0.21 % (0.70 % rotated)	98.42 % @ 7,771	QPS PQ, decisively

Memory triangle (1 M × 768, §11.3a):

Index	resident bytes	recall reached
bare IVF-PQ (no re-ranking)	92 MB	~62.6 %
OPQ+IVF-PQ+Refine	≈ 92 MB + 3,072 MB f32 copy ≈ 3.16 GiB	99.8 %
hnswlib / FAISS-HNSW	3.19 GiB / 3.19 GiB	99.8 %
QuIVer (BQ-native), with both repairs	675 MiB	88.99 % → 97.53 %

The honest reading: on collapse-tier data a PQ pipeline is *more accurate* than our best repair, and what our repair buys is not a win but the same recall band at **4.7×** less memory — a different point on the curve, not a free lunch. On the competitive tier (≥ 88 % recall) the BQ-native index wins outright and the memory advantage becomes a bonus rather than the whole argument.

9. Discussion

9.1 The decision a deployer actually faces

Combining §5–§8 into one table — every quantity is computable **before** building an index, in seconds:

sign_info	$\ \mu\ $ (headroom)	probe_ef (min of both metrics)	what to do	expected outcome
= 0.000 (sign plane dead)	any	—	rotate (§8.6) — task-preserving <i>and</i> stronger; centre only if rotation is not an option	GIST 2.10→ 60.22 % , SIFT 15.77→ 60.24 % @ef=64 (centring: 39.74 / 30.64); GIST becomes 1.3× faster than HNSW at 84 % recall
≥ 0.6 , $\ \mu\ \geq 0.3$, headroom > 5 pp	large	$\geq 50 \%$	centre as a cheap option	small-to-moderate gain (+0.3...+5.3 pp), never harmful in our 6-dataset set
≥ 0.6 but headroom ≤ 5 pp	large	$\geq 50 \%$	do not bother	Cohere: −1.2 pp (neutral) — the tier is already at 95 %
any	any	< 50 %	use a float32 index	the code cannot rank; the fault is not fixable by translation
sign_info = 1.000 but recall $\ll 50 \%$	~0	< 50 %	do not build any graph index on this task	index-agnostic collapse (Random-Sphere, Gaussian-960): HNSW also collapses (1.40 %)

And one row the gate deliberately does **not** cover, because it needs a competitor curve:

measured recall $\geq \sim 88 \%$ on a competitive embedding | — | — | **QuIVer is the right index**: 4.6–5.0× at matched recall vs the best HNSW | the only tier where the paper’s index actually wins (§5) |

9.2 Translation or rotation? The head-to-head

Both revive the sign plane; they differ in what *else* they change. We ran the comparison (§8.6, seeded random rotation, zero storage, no training):

tier	centring	rotation	reading
collapse (GIST / SIFT)	+37.6 / +14.9 pp @ef=64, but it replaces the task (centred-cosine)	+58.1 / +44.5 pp , GT untouched (99.87–100 % overlap)	rotation wins — stronger <i>and</i> it changes nothing but the code
usable (GloVe)	+3.3 pp alone; ceiling 93.32 % with weighted navigation	−0.6 pp alone; 88.45 % with weighted navigation (54.06 % @ef=64)	centring is better at the ceiling, rotation is better at low ef ; a wash in practice
competitive (Cohere)	−1.2 pp (neutral)	−4.8 pp	centring, or leave it alone

So the ordering is not “one dominates”: **for a dead sign plane, rotate; for a live sign plane, do not rotate**. Both choices are made by the same free statistic, so a deployer does not have to guess. What remains open is the *learned*-transform direction (L12): we tested one seeded **random** rotation, not OPQ-style learned ones, so 60.22 % / 60.24 % should be read as a **lower bound** on what a transform-side repair can do.

9.3 What this says about the published applicability claim

The paper’s four tiers are an **applicability** gradient (a description of achieved recall) and are read by practitioners as a **competitiveness** gradient. Our competitor map (§8.4) shows the two differ: before repair, three of the four tiers with competitor data were dominated by HNSW; after repair, one becomes parity and one becomes a genuine intersection — but the only tier with a clear win remains the competitive one. The practical statement we would put in a deployment guide is therefore narrower than the paper’s:

Repair comparison @ef_s=64 (cheap navigation)

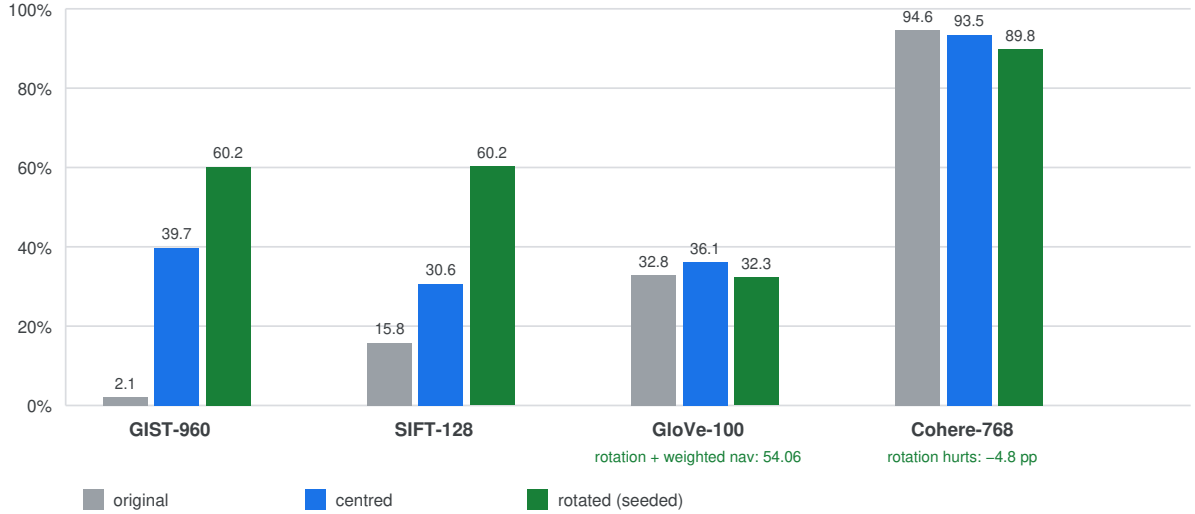


Figure 1. Repair comparison at $\text{ef}_s = 64$ (cheap navigation): original, centred and seeded-rotated R@10 for four tier representatives. Rotation is a pure gain only where the sign plane is dead (GIST-960 +58.1 pp, SIFT-128 +44.5 pp) and a loss on Cohere (−4.8 pp); centring is neutral-to-small elsewhere.

Use a BQ-native graph **when the embedding is competitive-tier** ($\geq \sim 88\%$ recall) and you **need memory/QPS**; on collapse-tier data, first check `sign_info` — if it is 0.000, centre it, knowing that this buys 2–5 pp of absolute recall but usually *not* the win.

9.4 A methodological note we owe the reader

Both the paper’s probe and our gate end up in the same place: **there is no single scalar that predicts recall**. Our two attempts to build one failed (§7.4), and the two strongest correlates we found (graph fidelity, `sign_info`) each have a counterexample that forbids a monotone formula. The deliverable is therefore a **triage rule plus a necessary condition**, with its abstention made explicit — which is, we suspect, the honest form of this kind of result, and the reason we report the failures alongside the wins.

9.5 Does the boundary appear outside the paper’s twelve datasets? (VIBE)

Table 11 is built from twelve datasets. We ran seven modern embedding benchmarks that the paper never evaluates (VIBE, `vector-index-bench/vibe`; 282 K – 3.9 M vectors, 768-d) to ask whether our triage rule extrapolates. It does — and the extrapolation is uncomfortable for the published claim:

VIBE row	$n \times d$	<code>sign_info</code>	<code>probe_ef</code> (min metric)	QuIVer @ $\text{ef}=64 \rightarrow$ @1024	hnswlib @ $\text{ef}=64$
<code>coco_nomic</code>	282 K $\times$ 768	0.382	0.6 % $\Rightarrow$ “use float32”	0.21 % $\rightarrow$ 3.59 %	86.23 % @17,053
<code>ccnews_nomic</code>	495 K $\times$ 768	0.690	99.0 % $\Rightarrow$ “usable”	25.65 % $\rightarrow$ 99.64 %	(not measured)

Two readings:

- **coco_nomic is a catastrophe of a kind the paper’s table cannot contain** — a modern, widely-used embedding benchmark on which the shipped index returns **0.21 %** R@10 while plain HNSW returns 86 % on the same task, with an exact scan at 100 % (neither the GT nor the task is at fault). Our gate catches

it before any index is built (`probe_ef` = 0.6 %), and **neither of our repairs rescues it** (rotation: 0.21 → 0.70 %): this is failure type (iii) of §6.4, where the honest advice is “change the code” (PQ: 98.42 %).

- **The probe estimates a ceiling, not a latency.** `ccnews_nomic` probes at 99.0 % while delivering 25.65 % at `ef=64`; its 99.64 % appears only at `ef_s=1024`. So “usable” must never be read as “fast at your operating point” — the §5 caveat restated in the one place where the two could be conflated.

Scope of this subsection: VIBE rows are an **extension**, not a reproduction (they are not in Table 11), and two of the seven rows were measured with the gate plus a single real-index sweep; the remaining five rows are reported with their `sign_info/probe` verdicts only (§11.3).

10. Limitations

All of the following are stated in the body text, not hidden in an appendix.

10.1 Platform and protocol

#	Limitation	Mitigation / why the conclusion survives
L1	Single machine, no AVX-512 (Intel i9-14900K; <code>Avx512F = false</code>) — the VPOPCNTDQ path QuIVer is designed around is never exercised	Recall-level conclusions are machine-independent. For speed we report the same-recall ratio, and note that our absolute MT-QPS is already 1.1–2.4× the paper’s (32 vs 16 threads), i.e. we are <i>not</i> measuring on a weakened platform. We do not assert the direction in which the ratio would move on Zen4.
L2	Metric deviation: the published SIFT/GIST rows are <i>Euclidean</i> , while the repository’s <code>prepare_all.py</code> normalises and recomputes GT by cosine, so our SIFT/GIST rows are cosine tasks	Disclosed in §4.2; our numbers are comparable to the paper’s <i>pipeline</i> , not to the public Euclidean leaderboards. All our <i>comparisons</i> put both arms on the same task, so the deviation cancels.
L3	Centring changes the similarity function: every “after” number in §8.2–§8.4 is Recall@10 on the <i>centred-cosine</i> task, not an improvement on the original task	Stated at the head of §8. Centring is also known post-processing (all-but-the-top family); we claim triage + quantification, not novelty of the transform. §8.6 covers this caveat by reporting a task-preserving alternative (seeded rotation), which on the collapse tier is both stronger and leaves the GT untouched (99.87–100 % overlap).
L4	The separability column (~5 % calibre uncertainty from <code>cos_std</code>) appears in the gate’s development tables	It is not used as a criterion in the final chain (§7.3), only reported for completeness.
L16	GT / metric alignment must be verified per dataset, not assumed. For Cohere the shipped ground truth is cosine while the on-disk f32 files are raw ($\ x\ \approx 13.8$), so any arm that does not re-normalise is silently scored against a different objective — this <i>inverted</i> our own first same-family measurement before it was diagnosed (§5.6b). The shipped check (<code>gt_metric_consistency_probe.py</code>) now makes normalisation a verified precondition for every arm.	

10.2 The reproduction has one unresolved row, one tie floor

#	Limitation
L5	RedCaps-1M (77.08 % vs 78.41 %) : the sampling protocol (which 1 M slice; whether self-matches are excluded) is unstated in the paper, and the four plausible readings span 7.4 pp . We report the reading that is closest to the paper and flag the row as partially reproduced.
L6	Wolt-CLIP has a tie floor: faiss_exact itself scores 90.09 % on this 1 M set, so the achievable ceiling is ~90 %. The “intersecting curves” verdict in §8.4 is therefore partly caused by ties, and this dataset’s recall must not be compared with other datasets’ (Cohere / GloVe / SIFT have faiss_exact $\geq$ 99.9 %). The centred variant used from §8.4 on is likewise tie-limited: exact scan 96.1 % (§5.6a, §8.7).

10.3 Experimental scope

#	Limitation
L7	Single M (32), single ef_c (128), single build per arm . Graph construction is concurrent and not bit-reproducible; we measured the spread over 3 independent builds $\times$ 5 representatives (§3.5): $\leq$ 0.25 pp at ef=64 (per dataset: GIST 0.22 / SIFT 0.25 / GloVe 0.10 / Wolt 0.19 / Cohere 0.16). Any graph-structure quantity (fidelity, CSR-derived) is a 256-node single-sample estimate.
L8	Our own two single-scalar predictors failed (§7.4). Reported as negative results; the delivered chain is triage + a necessary condition, and explicitly abstains on competitiveness.
L9	Memory footprint not reproduced $\rightarrow$ reproduced (§11.3a) : the paper’s “ $\approx 4.7\times$ less hot memory / < 1.3 GB per 1 M” is confirmed at 1:4.73 and 675 MiB per 1 M $\times$ 768 (hnsplib 3,193 MiB, FAISS-HNSW 3,189 MiB, IVF-Flat 2,949 MiB). Remaining caveat: we measured the index bytes , not peak RSS, and not the per-query hot-set growth.
L10	MSMARCO-5M scalability (Table 12) is out of scope — a 5 M-scale run is beyond this study’s budget, so the paper’s scaling claim is neither confirmed nor contradicted here.
L11	The multi-signal (TSNG) path was not A/B-tested under the navigation-metric switch (§8.3 covers the main path only).
L12	Only a random (seeded) orthogonal transform was tested (§8.6): we did not compare against learned rotations (OPQ-style) or other orthogonal transforms, and we do not claim rotation is optimal — only that it dominates centring on the collapse tier (60.22 % vs 39.74 % on GIST-960) while preserving the task. The row is therefore a <i>lower</i> bound on what a transform-side repair can achieve.
L13	Two baselines named by the paper could not be built on this machine (§5.3’s DiskANN-Rust and VSAG): DiskANN-Rust fails in openblas-src for want of a C/C++/BLAS toolchain (vcpkg, cmake, cl, gcc all absent), and the pyvsag wheel installed here is a Linux/cp310 build. These two arms are therefore missing, not measured as slow — a claim we deliberately do not make. Our competitor map covers hnsplib, FAISS-HNSW, USearch, IVF-Flat, faiss_exact , and (for quantizers) OPQ+IVF-PQ+Refine.

#	Limitation
L14	The quantizer comparison is one configuration family. Our PQ arm uses OPQ + IVF-PQ + exact f32 re-ranking; our RaBitQ arm is same-family (<code>IndexIVFRaBitQ</code> + f32 refine), with a normalised FastScan+SQ8 spot check used only as a recall-side consistency signal — neither reproduces the paper’s exact implementation, and the paper’s DiskANN PQ+FP and SSD arms are not run. The same-family arm also required a protocol correction (Cohere’s GT is cosine while its files are raw): its raw-vector numbers are retracted, and the corrected pair is in §5.6(b). Tier-jump PQ numbers were measured on six cells; they establish that PQ does <i>not</i> collapse on them, not that PQ dominates everywhere.
L15	The VIBE rows are an extension, not a reproduction (§9.5): they are not part of the paper’s Table 11, and of the seven VIBE benchmarks only two (<code>coco_nomic</code> , <code>ccnews_nomic</code>) received a full gate + real-index sweep; the other five carry <code>sign_info</code> /probe verdicts and their two navigation arms.

10.4 Statistical conventions used throughout

- Recall values are single-run unless labelled; the spread measured over **3 independent builds** is ≤ 0.25 pp, so differences below ~ 0.3 pp are not interpreted.
- Probe values (§7) always come with their sample size and the query-subset spread (4–7 pp); a single-run probe value is never quoted alone. The shipped table uses **K = 5 disjoint query subsets** (artifact field `probe_k`); re-running it at $K = 3$ changes no verdict and no value by more than 1.7 pp (§7.3).
- With the navigation switch **off**, frozen recall values reproduce within ≤ 0.17 pp (Cohere 97.52 / GIST-960 2.79 / GloVe-100 45.60), which is the regression guard for all **src/** work in §8.3.
- Competitor QPS numbers are quoted with the thread count that produced them; the competitor map is **thread-aligned at 32** (§11.3), because QPS at 16 vs 32 threads differs by up to $\sim 2\times$.

11. Artifacts and Reproduction

Every number in §4–§9 was produced by the code below on the machine described in §4.2, with `RUSTFLAGS="-C target-cpu=native"` and `--features ablation`. The **raw stdout logs** those runs produced are shipped in `results/logs/` (290 files, one README mapping prefixes to sections), so a clone of the artifact repository is sufficient to check any number quoted above.

11.1 Index-side harness (Rust bench, no library changes except the Sec. 8.3 switch)

Artifact	Purpose
<code>benches/bench_t2_b2_partitioned.rs</code>	the main arm: partitioned BQ search, single-arm mode (recall + MT-QPS + build time), frozen-value guard (<code>T2_FROZEN_RECALL</code>)
<code>benches/bench_t2_build_recon.rs</code>	exports the L0 CSR for graph-fidelity measurement (§6.3)
<code>src/index/bq.rs</code> , <code>src/index/quiver.rs</code> , <code>src/tsng.rs</code>	the only library change in this work: <code>TRIVIUM_NAV_WEIGHTED</code> (default off $\Rightarrow$ behaviour bit-identical; regression guard as above). Shipped as <code>patches/f1-nav-weighted.patch</code> for upstream review

11.2 Data preparation and analysis scripts (Python; `src/` untouched)

Script	Produces
<code>scripts/prepare_all.py</code>	the 12 benchmarking datasets (HF/ann-benchmarks sources), normalised + cosine GT
<code>scripts/research/gist960_collapse_prepare.py</code>	centred variants of any dataset (<code>--center <prefix>:<dim></code>), with the $\ \mu\ \leq 1$ guard of §8.1

Script	Produces
scripts/research/bq2_code_ceiling.py	the analytic code-only ceiling; <code>verify_identities()</code> asserts <code>analytic == bitwise</code> against <code>src/index/bq.rs</code>
scripts/research/deployability_gate.py	the 22-arm gate table of §7.3 (<code>sign_info</code> , $\ \mu\ $, <code>probe10</code> , <code>probe_ef</code> , <code>verdict</code>)
scripts/research/compat_probe_scaling.py	sample-size / query-subset sensitivity of the published probe (§7.2)
scripts/research/graph_neighbor_quality.py	$L0 \cap \{\text{cos, weighted, cheap}\}_{\text{top64}}$ (§6.3)
scripts/research/baseline_competitors.py	<code>hnswlib</code> / <code>FAISS-HNSW</code> / <code>Usearch</code> / <code>IVF-Flat</code> / <code>faiss_exact</code> curves on the same task (§5, §8.4); <code>BL_SKIP</code> can drop an arm
scripts/research/ivf_recall_diagnostic.py	the wrapper/parameter audit of §5.6(b): does <code>nprobe</code> reach the wrapped base, what does <code>k_factor</code> actually do, where does the exact arm saturate
scripts/research/gt_metric_consistency_probe.py	the GT/vector metric check of §5.6(b): which exact search reproduces the shipped ground truth
scripts/research/faiss_refine_mechanics_probe.py	the synthetic mechanics probe of §5.6(b): refine metric, default <code>k_factor</code> , monotonicity on a self-consistent task

11.3 Where each table comes from

Paper table	Source of record
§4.3 (12 rows)	<code>results/t2/*</code> per-dataset bench logs; frozen values in <code>docs/research/HANDOFF.md</code>
§5.3 (tier $\times$ competitor map)	<code>results/baseline/competitors_*.json</code>
§6.3 (graph fidelity)	<code>results/t2/gist960_collapse/graph_quality_*.json</code> (9 files; fields <code>overlap_cos_pct</code> / <code>overlap_w_pct</code> / <code>overlap_c_pct</code>)
§7.2–7.3 (gate + probe sensitivity)	<code>results/t2/deployability_gate.json</code> , <code>results/t2/p2_probe_scaling.json</code>
§8.2–8.3 (centre + navigation A/B)	<code>results/t2/gist960_collapse/*.json</code>
§8.4 (7-arm map)	<code>results/baseline/competitors_*.json</code>
§8.6 (rotation)	derived <code>*r_*.f32/i32</code> sets (<code>gist960r</code> / <code>sift128r</code> / <code>glove100r</code> / <code>coherer</code> , produced by <code>gist960_collapse_prepare.py --rotate</code> , with the GT-invariance check) + <code>results/logs/p4a_*.log</code> ; diagnosis in <code>docs/research/p4a-rotation-vs-centering.md</code>
§10-L9 (memory)	<code>results/t2/memory_footprint_cohere.json</code> (script <code>scripts/research/memory_index_footprint.py</code> ; log <code>results/logs/p4b_memory_cohere.log</code>)

11.3a Memory footprint (1 M $\times$ 768, M = 32)

Implementation	index bytes	B/vector	vs QuVer
QuIVer (2-bit BQ)	675 MiB	707.8	1.00 $\times$ (paper claims < 1.3 GB / 1 M: confirmed)
<code>hnswlib</code>	3,193 MiB	3,348.2	4.73$\times$ (paper claims 4.7 $\times$: confirmed)
<code>FAISS-HNSW</code>	3,189 MiB	3,344.1	4.72 $\times$
<code>FAISS IVF-Flat</code> (<code>nlist</code> $\approx$ 4 k)	2,949 MiB	3,092.3	4.37 $\times$
Bare IVF-PQ (<code>nlist</code> = 1,024, <code>m</code> = 96, no rerank)	92 MiB	96.5	0.14 $\times$ — but its recall caps at 62.6 %
OPQ+IVF-PQ+Refine (PQ codes + f32 rerank copy)	$\approx$ 3.16 GiB (92 MiB + 3,072 MiB)	3,312	4.8 $\times$ — needed to reach 99.8 %

$\Rightarrow$ The quantizer comparison sharpens the memory claim: **on PQ, cheap memory and high recall are mutually exclusive** (bare PQ is 92 MiB at 62.6 %; the refined pipeline is 3.16 GiB at 99.8 %). Only the BQ-native index gives 675 MiB **and** 99.8 % at the same time (§5.6 / §11.3b).

Measured as index bytes (bench-reported `Hot` for QuIVer; `save_index` size for `hnswlib`; `serialize_index` length for `FAISS`), not peak RSS. The 2-bit code accounts for ~ 183 MiB of QuIVer’s 675 MiB ($0.25 \text{ B/dim} \times 768 \times 1 \text{ M}$); the rest is graph + bookkeeping, and the per-dimension increment across our 128/256/512/768-dim arms ($522 \rightarrow 553 \rightarrow 614 \rightarrow 675$ MiB, $\approx 0.25 \text{ B/dim}$) matches 2 bits per dimension exactly. The repair arms (`c` / `r`) keep the same footprint (720 MiB on `gist960` and `gist960r` alike).

11.4 One-command reproduction of the central claim

1. build

```
RUSTFLAGS="-C target-cpu=native" cargo build --release --features ablation
```

2. dataset + centred variant (GIST-960 as the example)

```
python scripts/prepare_all.py gist960
```

```
python scripts/research/gist960_collapse_prepare.py --center gist960:960
```

3. before / after (single arm, includes the frozen-value guard)

```
T2_PREFIX=gist960 T2_DIM=960 T2_FROZEN_RECALL=2.79 cargo bench --features ablation --bench bench_t2_b2_p
```

```
T2_PREFIX=gist960c T2_DIM=960 cargo bench --features ablation --bench bench_t2_b2_partitioned
```

4. the gate (seconds, no index)

```
python scripts/research/deployability_gate.py
```

Expected: GIST-960 R@10 @ef=64 $\approx$ 2.1 % before and $\approx$ 39.7 % after centring (the paper's row is 2.01 %); the gate reports sign_info = 0.000 for gist960 and 0.981 for gist960c.

11.5 Upstream material (parallel track)

Artifact	Content
docs/research/upstream-issues.md	the full audit of the upstream reports (Chinese), including the one we retracted ourselves
docs/research/upstream-issue-draft.md	the paste-ready English version of those reports (five issues; not yet filed)
patches/f1-nav-weighted.patch	the §8.3 switch as a reviewable patch (default off)
docs/research/*.md	the full audit trail: every claim, its evidence, and the conclusions we retracted (three of our own)

11.6 Artifacts added after the first draft (second repair, quantizer arm, external validity)

All library changes remain **default-off and bit-identical when unset**; src/ now carries two such switches (TRIVIUM_NAV_WEIGHTED from §8.3, TRIVIUM_SIGN_ROTATE from §8.6), plus 2 new unit tests (152 library tests pass with both off and on).

§	Claim	Script	Product
8.6 (rotation, data side)	rotation preserves the task (GT overlap 99.87–100 %) and beats centring on the collapse tier	scripts/research/gist960_collapse_prepare.py --rotate/--rotate-center	src/train.f32 + results/logs/p4a_*.log
8.6 (rotation, engine side)	the same repair as one env var on the original files/GT	src/index/bq.rs (TRIVIUM_SIGN_ROTATE) + bench_t2_b2_partitioned	results/logs/p5_*.log
5.6 / 8.7 (quantizers)	PQ does not collapse where the 2-bit code does (six cells); QuIVer is 3.7 $\times$ faster at $\geq$ 99 % on Cohere	scripts/research/pq_matched_recall.py benches/bench_baselines.py (mode A/B)	results/logs/p2/pq_matched_recall_{cohere,sift1b}.json (from results/logs/p7_pq_*.log), results/baseline/competitors_cohere.json
5.6(b) (RaBitQ same-family + the normalisation trap)	the raw-vector inversion, the parameter audit (k_factor), the GT-metric check, and the corrected (normalised) pair	scripts/research/bench_rabitq_refine.py (RA_CONTROL, RA_NORMALIZE), ivf_recall_diagnostic.py, gt_metric_consistency_probe.py, faiss_refine_mechanics_probe.py	results/p2/{rabitq_refine,ivfflat_control}_norm_cohere.json (raw- metric; kept as the trap record), (pyf record), ivf_recall_diagnostic_cohere.json, gt_metric_consistency_cohere.json, faiss_refine_mechanics_probe.json, rabitq_refine_gist960c.json

§	Claim	Script	Product
6.4(ii) (second witness)	every competitor collapses on Gaussian-960 (hnsplib / FAISS-HNSW / USearch / IVF-Flat)	scripts/research/baseline_competitors/baseline/competitors_gauss960.json (BL_SKIP optional)	
8.6 (engine-side rc)	both switches reproduce the data-side rc arm on gist960c (80.12 / 97.13)	bench_t2_b2_partitioned + TRIVIUM_SIGN_ROTATE + TRIVIUM_NAV_WEIGHTED	results/logs/p8_10_engine_rc.log (retry product, after the queue's first attempt failed)
P8 queue	the ten-step unattended batch behind the rows above	scripts/research/p8_queue.py, scripts/research/p8_collect_numbers.py	results/t2/p8_queue_summary.json, results/logs/p8_*.log (step-10 retry note: docs/research/p8-queue-report.md §4-D5)
6.5 (bit budget)	2-bit allocation vs 4-bit ceiling on the four collapse datasets	scripts/research/bit_budget_preval.py	results/t2/bit_budget_*.json
7.3 (prospective validation)	nav-metric rule held out on 6 arms: 6/6	scripts/research/nav_rule_validation.py	results/t2/nav_rule_validation.json
9.5 (external validity)	seven VIBE benchmarks; coco_nomic is a new catastrophe	scripts/research/download_vibe_benchmarks.py, scripts/research/convert_hdf5_to_gaf82ops.py	results/t2/deployability_gate.json
3.5 / 10.3 (noise floor)	3 builds $\times$ 5 datasets $\Rightarrow$ spread ≤ 0.25 pp	bench_t2_b2_partitioned re-runs	results/logs/p7q_*.seed{1,2,3}.log, results/t2/p7_stagec_report.json
7.3 (probe K = 5)	K = 3 $\rightarrow$ K = 5 changes no verdict, drift ≤ 1.7 pp	scripts/research/deployability_gate.py (K = 5), scripts/research/p7_k5_vs_k3.py	results/t2/deployability_gate.json, results/t2/p7_k5_vs_k3.json
11.3a (memory)	QuIVer 675 MiB vs refined PQ ≈ 3.16 GiB vs hnsplib 3.19 GiB	scripts/research/memory_footprint.py	results/t2/memory_footprint_cohere.json

Reproducing §8.6’s engine-side switch end to end:

```
TRIVIUM_SIGN_ROTATE=20260923 T2_PREFIX=sift128 T2_DIM=128 \
cargo bench --features ablation --bench bench_t2_b2_partitioned # 15.77 % -> 62.66 % @ef=64
```

12. References, Artifact Statement, and How to Assemble This Draft

Every entry below was checked against its published version on 2026-10-09 (arXiv ids, venues and author lists verified against the arXiv abstract pages). We list what we actually used, with the version we used.

12.1 The object of study

1. **QuIVer: Rethinking ANN Graph Topology via Training-Free Binary Quantization.** W. Xiao, Z. Wang, C. Li. arXiv:2605.02171 (2026), cs.DB. — the paper evaluated and extended throughout §§4–7.
2. **README_QUIVER.md** (shipped with the implementation). — dataset preparation, benchmark drivers, and the step-by-step reproduction guide this work follows. §4.2’s “one protocol deviation” is a deviation from *this* document, not from the paper text.
3. Repository README (README.md / README_EN.md) — the published claims about hot/cold memory separation, the dimension guidance (≤ 3072), and the TSNG research track. §5 and §11.3a check the memory claim; §10.3/L11 records that the TSNG path was not A/B-tested under our navigation switch.

12.2 Baseline implementations we measured against

#	Baseline	Reference / source
4	HNSW (algorithm)	Yu. A. Malkov, D. A. Yashunin. <i>Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs</i> . IEEE TPAMI 42(4), 2020. arXiv:1603.09320
5	hnsplib (implementation)	https://github.com/nmslib/hnswlib

#	Baseline	Reference / source
6	FAISS (library)	J. Johnson, M. Douze, H. Jégou. <i>Billion-scale similarity search with GPUs</i> . IEEE Trans. Big Data, 2019. arXiv:1702.08734; and M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou. <i>The Faiss library</i> . arXiv:2401.08281 (2025)
7	USearch (implementation)	https://github.com/unum-cloud/usearch
8	Inverted-file index (IVF / IVF-Flat)	J. Sivic, A. Zisserman. <i>Video Google: a text retrieval approach to object matching in videos</i> . ICCV 2003
9	Product quantization (PQ)	H. Jégou, M. Douze, C. Schmid. <i>Product quantization for nearest neighbor search</i> . IEEE TPAMI 33(1), 2011
10	Optimized PQ (OPQ)	T. Ge, K. He, Q. Ke, J. Sun. <i>Optimized product quantization for approximate nearest neighbor search</i> . CVPR 2013
11	RaBitQ	J. Gao, C. Long. <i>RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search</i> . SIGMOD 2024 (DOI 10.1145/3654970). arXiv:2405.12497
12	DiskANN / Vamana graph	S. Subramanya, et al. <i>DiskANN: Fast accurate billion-point nearest neighbor search on a single node</i> . NeurIPS 2019
13	FAISS IndexIVFRaBitQ / IndexRefineFlat / IndexPreTransform	FAISS 1.15 API (the same-family RaBitQ arm of §5.6(b), and the PQ arm of §5.6(a))

Not measured, and why: the paper’s DiskANN-**Rust** and VSAG arms (§10.3/L13) — a C/C++/BLAS toolchain is absent on the evaluation machine and the `pyvsag` wheel available there is a Linux/build-mismatched artifact.

12.3 Background and method references

- J. Mu, P. Viswanath. *All-but-the-Top: Simple and effective postprocessing for word representations*. ICLR 2018. arXiv:1702.01417 — the centring step of §8.1 is this post-processing, applied to the query and base vectors before quantization.
- VIBE: Vector Index Benchmark for Embeddings**. E. Jääsaari, V. Hyvönen, M. Ceccarello, T. Roos, M. Aumüller. arXiv:2505.17810 (2025); J. Data-centric ML Research (2026) — <https://github.com/vector-index-bench/vector-index-bench>. The seven external benchmarks of §9.5.
- Embedding models whose released vectors we use (Cohere-768, OpenAI-1536/3072, Nomic-768, Jina-768, DINOv2, MiniLM, BGE-M3, GloVe-100, Wolt-CLIP-512): cited via the dataset preparation scripts of (2) and (15), which name the model and revision for each row. We make no claim about the models themselves.

12.4 Artifact statement (what is ours, what is upstream, and the licence)

The upstream project is licensed **Apache-2.0** (LICENSE). This fork keeps that licence, and every change we made is confined to the following, so a reader can separate our work from upstream’s line by line:

Directory / file	Ours?	Content
<code>Cargo.toml</code> , <code>src/**</code> (except two switches) <code>src/index/bq.rs</code>	upstream ours, +2 switches	the Quiver implementation we evaluate TRIVIUM_NAV_WEIGHTED (§8.3) and TRIVIUM_SIGN_ROTATE (§8.6): both default off and bit-identical when unset , shipped as reviewable patches in patches/ , plus 2 new unit tests
<code>benches/bench_t2_b2_partitioned.rs</code>	upstream	the index-side harness we drive via <code>T2_*</code> environment variables (no source change needed)

Directory / file	Ours?	Content
benches/bench_baselines.py	upstream, +env overrides	grids/thread counts made overridable so the quantizer arm can be run at 32 threads (§10.4)
scripts/research/**	ours	index-free gate, competitor maps, rotation/centring preparation, PQ & RaBitQ arms, bit-budget probe, memory footprint, multi-seed spread, all report scripts
docs/paper/**	ours	this draft (§0–§12)
docs/research/**	ours	the audit trail: every claim with its evidence, including our own conclusions that were retracted along the way (most recently the raw-vector RaBitQ numbers, §5.6b)
results/**	ours (generated)	the citable store: every product the paper cites, plus the raw stdout logs under results/logs/ (290 files copied out of the local scratch directory, so a clone of this repository is self-sufficient for verification)

Traceability. Every number in this draft resolves to a file under **results/**** or a log under **results/logs/****, and §11.3 + §11.6 give the table → artifact mapping. Statements without an artifact are explicitly labelled as assumptions (e.g. the RedCaps row, §4.4).

Licence. This report is released under **Creative Commons Attribution 4.0 International (CC BY 4.0)**. The code, scripts and artifacts in the accompanying repository remain under the upstream **Apache-2.0** licence (LICENSE).

Reproduction entry points. §11.4 (the central claim, five commands) and §11.6 (the second repair, the quantizer arm, the external benchmarks). The full unattended batch that produced the last group of numbers is **scripts/research/p8_queue.py**, whose per-step log and summary (**results/t2/p8_queue_summary.json**) are part of the artifact.

What is *not* in the artifact. The upstream paper’s own source or LaTeX; the VIBE and embedding-model downloads (public, large, and pinned by their own repositories); and the two blocked baselines of §10.3/L13.